



Research paper

Situation-aware deep reinforcement learning for unmanned aerial vehicle swarm navigation in dynamic multi-obstacle environments

Hongwei Zhao^a, Ning Yu^a, Yanyan Zhao^a, Juan Feng^{b,*}, Yu Wang^c

^a School of Electronics and Information, Northwestern Polytechnical University, Xi'an, Shaanxi, 710129, China

^b School of Aerospace Science and Technology, Xidian University, Xi'an, Shaanxi, 710071, China

^c Yangtze normal university, Fuling, Chongqi, 408100, China



ARTICLE INFO

Keywords:

Artificial intelligence implementation
Autonomous navigation application
Hybrid path planning framework
Situation awareness
Deep reinforcement learning

ABSTRACT

Autonomous navigation for unmanned aerial vehicle (UAV) swarms in dynamic, obstacle-rich environments remains a significant challenge due to unpredictable obstacle movements and complex swarm coordination requirements. Current approaches exhibit critical limitations: traditional algorithms suffer from high computational complexity and poor adaptability, while reinforcement learning (RL) methods often converge to local optima and require extensive training. To address these issues, this study proposes a hybrid AI framework integrating Situation Awareness Path Optimization (SAPO) with an enhanced Deep Deterministic Policy Gradient (DDPG), termed SAPO-DDPG. The SAPO module employs a channel-attention-enhanced CNN-LSTM network for real-time spatiotemporal danger perception, directly tackling environmental unpredictability. Concurrently, the enhanced DDPG incorporates adaptive multi-objective reward formulation and prioritized experience replay to mitigate local optima convergence and accelerate training. Evaluated on realistic simulation datasets, the proposed method consistently outperforms the state-of-the-art TSEB-DDPG baseline. It achieves shorter path lengths (individual: 2.1%, swarm: 4.2% reduction) while guaranteeing near-zero collision navigation, demonstrating robust autonomous operation for UAV swarms in complex, dynamic, and cluttered environments.

1. Introduction

The application of unmanned aerial vehicle (UAV) swarms is rapidly expanding into complex domains such as search and rescue, infrastructure inspection, and precision agriculture. The global UAV market size has grown from USD 9.3 billion in 2013 to nearly USD 60 billion in 2023, reflecting a compound annual growth rate of 20.43% (Insights, 2023). This growth underscores the critical need for robust autonomous navigation systems capable of operating in dynamic, Global Positioning System (GPS)-denied environments cluttered with moving obstacles. However, achieving efficient, collision-free path planning for multiple UAVs under these conditions remains a significant challenge, as traditional reliance on pre-existing maps or direct human control proves inadequate due to signal limitations, limited fields of view, and slow response times (Hu et al., 2024a; Jiang and Huang, 2023; Huang et al., 2024; Shao et al., 2025; Sulieman et al., 2024; Rao et al., 2023; Fan et al., 2022).

Existing UAV swarm path planning methodologies can be broadly categorized, yet each exhibits critical limitations in dynamic settings (Hu et al., 2024a; Jiang and Huang, 2023; Huang et al., 2024;

Shao et al., 2025; Sulieman et al., 2024; Rao et al., 2023; Fan et al., 2022; Yu et al., 2023a; Tu et al., 2024; Pang et al., 2024; Wang et al., 2024; Wen et al., 2023; Tang et al., 2024b; Hu et al., 2025; Zhong et al., 2025; Boulares et al., 2024; Zhou et al., 2024; Xu et al., 2023b; Zhang et al., 2024; Tang et al., 2024a; Bo et al., 2024; Fan, 2023; Zhang et al., 2023; Saeed et al., 2024; Ajith and Jolly, 2023; Zhu and Ye, 2025). Traditional algorithms (e.g., A*, Rapidly-exploring Random Trees (RRT), Artificial Potential Field (APF)), while effective in static environments, suffer from high computational complexity and inherent rigidity. For instance, improved A variants reduce computation time but fail to address nonlinear robustness degradation at high path costs (Hu et al., 2024b). Goal-biased RRT accelerates initial path discovery but lacks real-time dynamic obstacle response (Liu et al., 2023), and bidirectional APF-RRT* exhibits weaknesses in multi-objective coordination and dynamic avoidance (Xu et al., 2023a). Intelligent optimization techniques (e.g., Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Ant Colony Optimization (ACO)) support multi-objective optimization (Yu et al., 2023a; Tu et al.,

* Corresponding author.

E-mail addresses: hongvi_zhao@126.com (H. Zhao), yuning715@mail.nwpu.edu.cn (N. Yu), zhaoyanyan@mail.nwpu.edu.cn (Y. Zhao), fengj@xidian.edu.cn (J. Feng), wangyu_827@foxmail.com (Y. Wang).

<https://doi.org/10.1016/j.engappai.2025.113518>

Received 10 May 2025; Received in revised form 13 October 2025; Accepted 8 December 2025

Available online 13 December 2025

0952-1976/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1

The performance comparison and applicability analysis of various methods.

Methodologies	Algorithm	Dynamic response	Multi-objective optimization	Computational efficiency	Global Map Required	Application scenario
Traditional methods	Graph search(e.g. A*)	Low	Single-Objective	Low	Yes	Static small-scale environment
	Sampling(e.g. RRT)	Medium	Local optimization	Medium	No	3D exploration in unknown spaces
	Artificial potential field	Medium	Local optimization	High	No	Real-time collision avoidance
Intelligent optimization techniques	Genetic algorithm	Low	Multi-objective	Medium	No	Offline multi-constraint planning
	Particle swarm optimization	Medium	Multi-objective	High	No	Fast planning in static obstacles
	Ant colony optimization	Medium	Multi-objective	Low	No	Multi-obstacle environment
Deep reinforcement learning	DQN	High	Partial Support	Low (training)	No	Dynamic unknown environments
	PPO	High	Multi-Objective	High (execution)	No	High-dynamic swarm coordination

2024; Pang et al., 2024; Wang et al., 2024; Wen et al., 2023; Tang et al., 2024b; Hu et al., 2025; Zhong et al., 2025) but incur prohibitive computational overhead. Methods like the Expert GA (Fan et al., 2024) introduce decision instability, while ACO variants often neglect dynamic obstacles, creating safety vulnerabilities (Yu et al., 2023b). Deep Reinforcement Learning (DRL) methods (Boulares et al., 2024; Zhou et al., 2024; Xu et al., 2023b; Zhang et al., 2024; Tang et al., 2024a; Bo et al., 2024) enable adaptive planning but face unresolved challenges. Models based on Deep Q-Network and Artificial Potential Field (B-APFDQN) (Kong et al., 2023) improve training efficiency but suffer from “black-box” interpretability, and while recent algorithms like Soft Actor–Critic (SAC) (Zhao et al., 2024; Zhong et al., 2024) and Twin Delayed Deep Deterministic policy gradient (TD3) (Luo et al., 2024) show promise, they often lack validation in dynamic swarm settings. The Three-Step Experience Buffer (TSEB) Deep Deterministic Policy Gradient (DDPG) model (Tian et al., 2024), though advanced, demonstrates severe generalization limitations. Collectively, these limitations manifest as three core deficiencies: weak real-time performance, inadequate global planning capability, and insufficient robustness in dynamic obstacle avoidance.

The characteristics and application scenarios associated with these methods are detailed in Table 1.

To overcome these limitations, this paper proposes a hybrid framework that integrates Situation Awareness Path Optimization (SAPO) (Yu et al., 2024) with an enhanced Deep Deterministic Policy Gradient (DDPG) algorithm, called SAPO-DDPG. The framework's components are designed to address the specific shortcomings of prior works. First, a Situation Assessment Module eliminates global map dependency by constructing real-time spatiotemporal danger models through a channel-attention-enhanced Convolutional Neural Network with Long Short-Term Memory networks (CNN–LSTM) (Kumar et al., 2024), directly enhancing real-time responsiveness. Second, a Multi-objective Reward Formulation combined with SAPO's path optimization reconciles conflicting objectives (safety, efficiency, smoothness, swarm coordination), significantly boosting global planning capability. Third, Prioritized Experience Replay (Schaul et al., 2016) and our novel Adaptive Harmonic Analysis of NDVI Time-Series (HANTs) Filtering (AHF) algorithm—an enhancement of the HANTs filter (Aliabad et al., 2024; Zhao et al., 2025)? — accelerate DDPG convergence and improve sample efficiency. For instance, in simulated tests within a 500 m × 300 m dynamic environment, this synergy enabled a 5-UAV swarm to achieve a mean path length of 1619.5 m with near-zero collision probability, outperforming the TSEB-DDPG baseline.

The main contributions of this work are summarized as follows:

We propose a novel hybrid architecture (SAPO-DDPG) that synergistically combines real-time spatiotemporal perception with adaptive decision-making, actively preventing local optima convergence and ensuring globally smooth paths via integrated curve optimization.

We introduce a key algorithmic innovation by categorizing environmental states before policy training, enabling dynamic model integration and continuous fine-tuning during deployment without costly policy network reinitialization.

Table 2

Table of notations in Section 2.

Symbol	Description
$x[n]$	Original time series data
$\hat{x}[n]$	Reconstructed time series after filtering
$x'[n]$	Updated sequence after outlier replacement
$X(e^{j\omega})$	Frequency domain representation after DTFT
K	Error threshold for outlier detection
N_e	Maximum number of deletion points per iteration
f_n	Number of low-frequency components for reconstruction
$L(x[n])$	Reconstruction loss function (Mean Squared Error)
M	Maximum value of the sequence $x[n]$ (for normalization)
N_I	Number of iterations until convergence
L_{f_n}	Minimum loss value for frequency number f_n
Θ	AHF parameter set: $\Theta = [K, N_e, f_n]$
$\nabla_{\Theta} L$	Gradient of loss with respect to AHF parameters

We develop the novel Adaptive HANTs Filtering algorithm, which dynamically adjusts filtering parameters via iterative loss minimization, significantly boosting the accuracy of environmental state classification for safer path planning.

The remainder of this article is organized as follows. Section 2 presents the structure of the AHF model. In Section 3, we discuss the underlying principles of the SAPO model and the DDPG algorithm. Section 4 outlines the hybrid path planning model for UAV swarms based on the SAPO-DDPG. Experiments and results are detailed in Section 5, while conclusions are provided in Section 6.

2. The structure of the AHF model

The notations used in this section are shown in Table 2.

The HANTs (Aliabad et al., 2024) is a filtering algorithm grounded in the harmonic analysis of time series, which combining smoothing and filtering. This approach leverages the temporal and spatial characteristics of sensor data, connecting spatial distribution patterns with temporal variations. The core of the algorithm employs Fourier transform and least-squares fitting to decompose time-series data into a superposition of sinusoidal and cosine curves with distinct phases, frequencies, and amplitudes. A subset of these curves, which accurately captures the essential features of the original series, is selected to reconstruct the filtered time-series data.

As an enhancement of the Fast Fourier Transform (FFT), the HANTs algorithm can eliminate high-frequency noise and accommodates non-uniformly sampled time-series data. Additionally, it serves three key functions: (1) gap-filling and reconstruction of missing data, (2) data compression, and (3) smoothing of irregular time-series.

The specific implementation process of the algorithm is outlined as follows:

i. A Fourier transform $DTFT\{x[n]\}$ is applied to the time series of each pixel point. As shown in Eq. (1):

$$X(e^{j\omega}) = DTFT\{x[n]\} = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n} \quad (1)$$

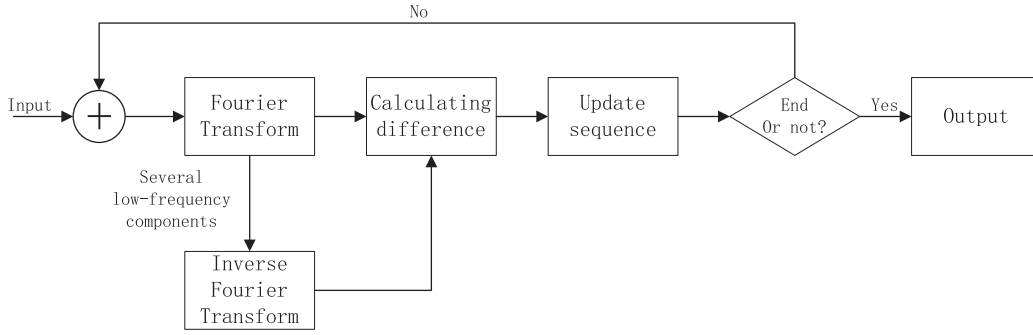


Fig. 1. The algorithm flow chart of the HANTs filter model.

Table 3
Filtering algorithms comparison.

Algorithm	Advantages	Disadvantages
Moving Average	– Simple implementation – Low computation cost	– Causes phase lag – Poor step response
Exponential Smoothing	– Weighted for recent data – Computationally efficient	– Parameter tuning needed – Lag for abrupt changes
Kalman Filter	– Optimal linear estimation – Combines prediction/update	– Needs accurate model – Degrades for nonlinear systems
Wavelet Filter	– Time–frequency localization – Preserves signal features	– Basis selection critical – Computationally intensive
HANTS Filter	– Handles non-uniform sampling – Reconstructs missing data	– Parameter setting sensitive – Computationally heavy
Particle Filter	– Handles nonlinear systems – Suitable for complex distributions	– Very high computation cost – Particle degeneracy problem
Median Filter	– Excellent for impulse noise – Preserves signal edges	– Ineffective for Gaussian noise – Causes signal distortion

ii. Several low-frequency components are selected for inverse Fourier transform $IDTFT\{X(e^{j\omega})\}$, which produces a new sequence. As shown in Eq. (2). The contribution coefficients of each sine or cosine function in the reconstructed original data set are obtained through least squares curve fitting.

$$\hat{x}[n] = IDTFT\{X(e^{j\omega})\} = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\omega}) e^{j\omega n} d\omega \quad (2)$$

iii. The difference between the original time series $x[n]$ and the reconstructed series $\hat{x}[n]$ is then calculated. If this difference exceeds a predetermined threshold K , the point is identified as an outlier, removed from the original sequence, and replaced with the corresponding value from the new sequence. As shown in Eq. (3).

$$x'[n] = \begin{cases} \hat{x}[n], & \text{if } |x[n] - \hat{x}[n]| > K \\ x[n], & \text{if } |x[n] - \hat{x}[n]| < K \end{cases} \quad (3)$$

iv. This process is repeated for the original sequence of changes until no cloud-contaminated points remain or a predetermined end condition is met.

These equations form the HANTs filtering core: Equation (1) performs spectral decomposition, Eq. (2) reconstructs from low frequencies, and Eq. (3) detects and replaces outliers. This iterative process removes noise while preserving essential signal features.

The algorithm flowchart is shown in Fig. 1. Conceptually, HANTs effectively approximates the original data using a limited set of low-frequency components, achieving a balance between eliminating significant noise and preserving essential features.

Table 3 shows the advantages and disadvantages of the commonly used filtering algorithms at present (Zhao et al., 2025).

The HANTs filter belongs to harmonic analysis-based smoothing techniques and differs from conventional static filters (e.g., moving average) in three key aspects:

- **Non-uniform sampling:** Unlike Fourier transforms requiring equidistant intervals, HANTs handles irregular time-series data through least-squares fitting, critical for UAVs with intermittent sensor updates.
- **Feature preservation:** While static low-pass filters indiscriminately suppress high frequencies, HANTs selectively reconstructs components through harmonic superposition, preserving abrupt environmental changes.
- **Data imputation:** HANTs inherently repairs missing data (common in GPS-denied areas) by replacing outliers with fitted values, whereas static filters require separate preprocessing.

A limitation of the standard HANTs filter is its reliance on fixed parameters during iteration, which demands accurate initial settings and cannot adapt to varying filtering requirements across different processing stages. To address this, we propose the Adaptive HANTs Filtering (AHF) model, which enhances dynamic adjustment and iteration efficiency by replacing static parameters with adaptive, functionally determined equivalents.

First, we introduce a loss function to calculate the difference between the sequence before and after the iteration. As shown in Eq. (4):

$$L(x[n]) = \frac{1}{n} \sum_{i=1}^n (x'[i] - x[i])^2 \quad (4)$$

From this, the error threshold K , can be defined as shown in Eq. (5):

$$K = \frac{1}{M} \sqrt{L(x[n])} \quad (5)$$

where M is the maximum value of the sequence $x[n]$. The maximum number of deletion points N_e can be obtained by calculating the difference between the loss function in two successive iterations, as shown in Eq. (6):

$$N_{ei} = \frac{L_{i-1}(x[n])N_{e(i-1)}}{L_i(x[n])} \quad (6)$$

where N_{ei} is the maximum number of deletion points in the i_{th} iteration, $L_i(x|n)$ is the loss function in the i_{th} iteration, $N_{e(i-1)}$ is the maximum number of deletion points in the $(i-1)_{\text{th}}$ iteration, and $L_{i-1}(x|n)$ is the loss function in the $(i-1)_{\text{th}}$ iteration. Finally, the low frequency number f_n can be updated based on the trend of the reconstruction loss, as shown in Eq. (7):

$$f_{n+} = \begin{cases} 0, & \text{if } \frac{L_{i-1}(x[n]) - L_i(x[n])}{L_{i-2}(x[n]) - L_{i-1}(x[n])} > 0 \\ -1, & \text{if } \frac{L_{i-1}(x[n]) - L_i(x[n])}{L_{i-2}(x[n]) - L_{i-1}(x[n])} \leq 0 \end{cases} \quad (7)$$

When f_n changes, the minimum loss function of the optimal sequence corresponding to f_n compared with the original sequence will be recorded and recorded as L_{f_n} . When L_{f_n} cannot obtain the minimum value when f_n changes, the iteration ends, and the optimal sequence corresponding to the minimum value of L_{f_n} is taken as the output result.

The AHF enhancement introduces dynamic parameter adaptation. Eqs. (5)–(7) automatically adjust filtering parameters based on reconstruction loss, eliminating manual tuning while balancing noise suppression and feature preservation across processing stages.

This method satisfies optimization equivalence.

The parameter update rules (Eq. (5)–(7)) implement stochastic gradient descent (Eq. (8)):

$$\nabla_{\Theta} L = \left[\frac{\partial L}{\partial K}, \frac{\partial L}{\partial N_e}, \frac{\partial L}{\partial f_n} \right]^T, \quad \Theta = [K, N_e, f_n] \quad (8)$$

Minimizing reconstruction loss $L(x[n])$ (Eq. (4)) ensures AHF converges to the optimal filter configuration.

The relationship between the parameters is shown in Fig. 2.

In comparison to the original algorithm, the AHF model enhances iteration by gradually updating parameters throughout the process. This not only allows for the preservation of more feature information during the initial iterations but also accelerates the convergence speed of the iterations. Additionally, it minimizes the impact of human factors on the model.

The proposed AHF model eliminates manual tuning by introducing dynamic parameter adaptation, contrasting with existing adaptive filters:

- **Vs. Kalman variants:** While adaptive Kalman filters adjust noise covariance matrices, they assume linear dynamics (violated by UAV maneuvers). AHF's loss-driven updates are model-agnostic.
- **Vs. wavelet denoising:** Wavelet methods adapt thresholds per frequency band but cannot impute missing data. AHF jointly optimizes smoothing and gap-filling.
- **Vs. RL-based filters:** Reinforcement learning adapters incur high latency. AHF's closed-form updates ensure real-time operation (< 1 ms per iteration).

Therefore, the AHF model can serve as an independent and versatile module suitable for a broader range of time-series smoothing and filtering applications.

3. The SAPO model and the DDPG algorithm

The notations used in this section are shown in [Tables 4 and 5](#).

3.1. The improved SAPO model

The SAPO model is a path optimization method that leverages situation awareness and assessment techniques to integrate information effectively. It evaluates path quality through spatio-temporal analysis and ultimately employs a curve fitting method to optimize the path. This approach challenges the conventional shorter-path-better notion

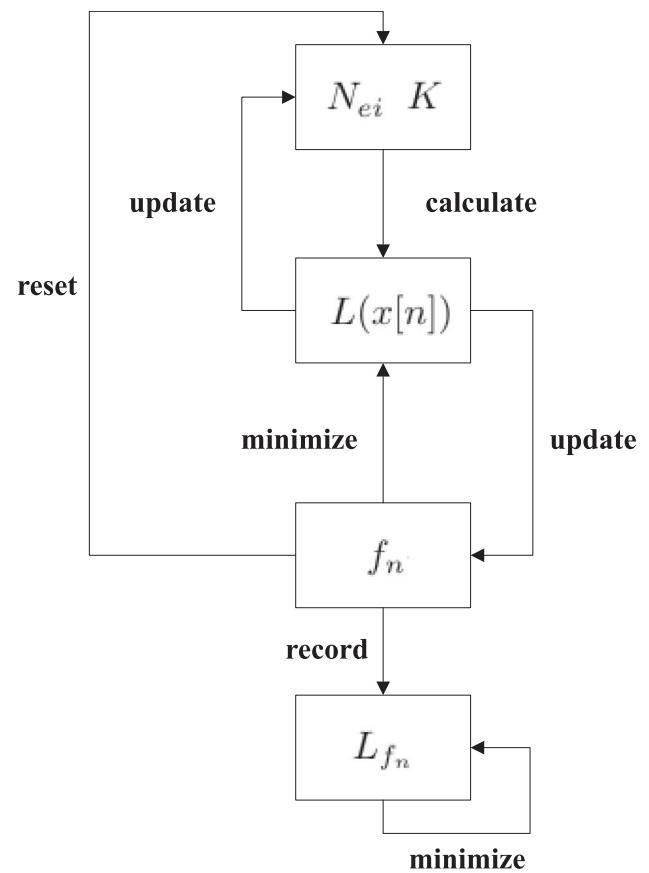


Fig. 2. The relationship between parameters in AHF model.

Table 4
Table of notations in Section 3.1.

Symbol	Description
F_c, F_s, F'_c	Feature maps in channel attention mechanism
C, S	Channel dimension and reduced dimension in attention
$f(x)$	Gaussian probability density function for danger field
$L(\mu, \sigma^2)$	Likelihood function for parameter estimation
$l(\mu, \sigma^2)$	Log-likelihood function
$\hat{\mu}, \hat{\sigma}^2$	Maximum likelihood estimates of Gaussian parameters
F	Path quality evaluation function
l	Path length in quality evaluation
N	Number of break points along the path
θ_x	Turning angle at break point x
ρ	Path feasibility probability
$D(v)$	Variance of average passing speed
n_u	Minimum number of feasible cross sections
m'	Number of UAV members in swarm subset
ω_j	Weight parameters in path quality function
Z	Path quality evaluation coefficient for multi-bend scenarios
v_0	Initial speed at area entry
v_m	Maximum UAV flight speed
r	Maximum radius of curvature
φ	Endpoint tangent angle
k	Curvature ($k = 1/r$)
L	Curve length in cyclotron curve
A	Constant in cyclotron curve equation $kL = A^2$

by considering multiple objectives including path length, smoothness, passage time, and maximum turning angle. Additionally, the method facilitates the integration of multi-source information, allowing data from different sensors to be cohesively combined, thereby providing more comprehensive support for path planning.

The SAPO model consists of two primary components: the situational assessment module, which is responsible for the integration

Table 5

Table of notations in Section 3.2.

Symbol	Description
$\pi(s \theta^\pi)$	Actor policy network
$Q(s,a \theta^Q)$	Critic value network
$\pi'(s \theta^{\pi'})$	Target actor network
$Q'(s,a \theta^{Q'})$	Target critic network
θ^π, θ^Q	Online network parameters
$\theta^{\pi'}, \theta^{Q'}$	Target network parameters
α	Learning rate
γ	Discount factor
N	Number of training samples in batch
L_c	Critic loss function
y_i	Target Q-value
R	Overall reward function
R_i	Sub-objective reward functions
ω_i	Weight parameters in reward function
δ_j	TD-error for experience j
P_j	Sampling probability in prioritized experience replay

of information, and the path optimization module, which focuses on optimizing processes. Each of these components is detailed in the subsequent sections.

3.1.1. Situation assessment module

Situation assessment encompasses three essential components: situational awareness, situational understanding, and situational prediction. Situational awareness involves gathering environmental information, such as maps and location data, through various sensors. Situational understanding focuses on analyzing this data and monitoring movement patterns. Finally, situational prediction merges this understanding with the capability to foresee future changes, applying constraints to maintain stability and safety.

Among them, situational awareness and understanding encompasses three primary components: information acquisition, target recognition, and target localization. Information acquisition primarily relies on visual input, leveraging multi-source fusion to generate a comprehensive map and location data essential for target recognition. Target recognition involves matching features such as contours and textures in image data to identify targets once a predetermined matching threshold is achieved. Finally, target localization employs multiple sensors to ascertain the target's position and movement by integrating map information with positional data.

The original model employs a CNN-LSTM architecture (Kumar et al., 2024) for feature extraction and processing. This hybrid network synergistically combines the CNN's strength in extracting local spatial features with the LSTM's proficiency in modeling temporal dependencies and managing long-term memory in time-series data. To enhance the efficiency of feature extraction, we have refined the original model by introducing a channel attention mechanism following the LSTM. This enhancement allows the network to focus on and learn more relevant features. The specific structure of the CNN-LSTM model is illustrated in Fig. 3.

The CNN component comprises input normalization (Eq. (9)), convolutional layers with ReLU activation (Eq. (10)), pooling layers for dimensionality reduction, and fully connected layers for feature integration. As illustrated in Fig. 4, after undergoing several convolution operations, the amount of information extracted from a single element in the data significantly increases.

$$x'_{ij} = \frac{x_{ij}}{\max \{x_{11}, \dots, x_{1n}, x_{21}, \dots, x_{mn}\}} \quad (9)$$

x_{ij} is the original data and x'_{ij} is the normalized data.

$$\text{ReLU}(x_{ij}) = \begin{cases} 0, & \text{if } x_{ij} < 0 \\ x_{ij}, & \text{if } x_{ij} \geq 0 \end{cases} \quad (10)$$

The LSTM network (Fig. 5) processes temporal sequences through gate mechanisms (input, forget, output gates) to capture essential long-term dependencies in sequences.

Recognizing that different features contribute differently to the model's performance, we incorporate a channel attention mechanism to enhance the network's ability to focus on and learn more relevant features. This module is illustrated in Fig. 6. Initially, a feature map $F_c = C \times 1 \times 1$ is generated using global mean pooling. Subsequently, we derive the feature $F_s = S \times 1$ by reducing the dimensionality of F_c . Once we have the two-dimensional feature map, we apply convolutional dependencies to capture the interactions among each channel and its neighbors, resulting in the transformed feature map $F'_c = C \times 1 \times 1$. Finally, we obtain the weight of each channel using the Sigmoid function.

The CNN-LSTM model is effective to achieve situational awareness and comprehension of the surrounding environment. Building on the perceived outcomes, further development of situation prediction is essential. This process primarily encompasses a classification problem and the resolution of a probability distribution issue. The environment can be categorized into three areas: safe, dangerous, and unknown. The safe zone represents areas that can be traversed freely, whereas the dangerous zone indicates regions that are obstructed or lack the conditions for passage. Due to observational limitations, certain areas cannot be definitively classified as safe, leading to their designation as unknown areas. To quantify the reliability of these unknown regions, we can define the security level of each point within them using a probability density function. Additionally, the passability of the unknown regions can be represented through a probability distribution function.

There is a clear relationship between the safety level of the unknown zone and its distance from the danger zone. As this distance increases, the likelihood of encountering danger diminishes gradually. Consequently, it can be assumed that the probability of danger and the corresponding distance are normally distributed, as illustrated in Eq. (11):

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (11)$$

where σ is the scale parameter and μ is the positional parameter. This Gaussian model probabilistically represents danger distribution around obstacles. Parameters μ and σ control the danger concentration and spatial decay, creating smooth gradients that facilitate stable optimization rather than binary constraints.

However, the impact of various obstacles on the surrounding environment varies, indicating that the parameters within the probability density function are not uniform. Therefore, it is essential to estimate these parameters based on available information in order to derive the appropriate probability distribution function. We employ the maximum likelihood estimation method for parameter estimation. Initially, we formulate the maximum likelihood function, as illustrated in Eq. (12):

$$L(\mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}} \quad (12)$$

Then take the logarithm of the likelihood function to get the log-likelihood function $l(\mu, \sigma^2)$ and take the derivative of it, as shown in Eq. (13)–(14):

$$\frac{\partial l(\mu, \sigma^2)}{\partial \mu} = \frac{\partial \ln L(\mu, \sigma^2)}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) \quad (13)$$

$$\frac{\partial l(\mu, \sigma^2)}{\partial \sigma^2} = \frac{\partial \ln L(\mu, \sigma^2)}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2(\sigma^2)^2} \sum_{i=1}^n (x_i - \mu)^2 \quad (14)$$

Set the derivative of the log-likelihood function to 0 to obtain the sample mean $\hat{\mu}$ and sample variance $\hat{\sigma}^2$, as shown in Eq. (15):

$$\begin{cases} \hat{\mu} = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \\ \hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \end{cases} \quad (15)$$

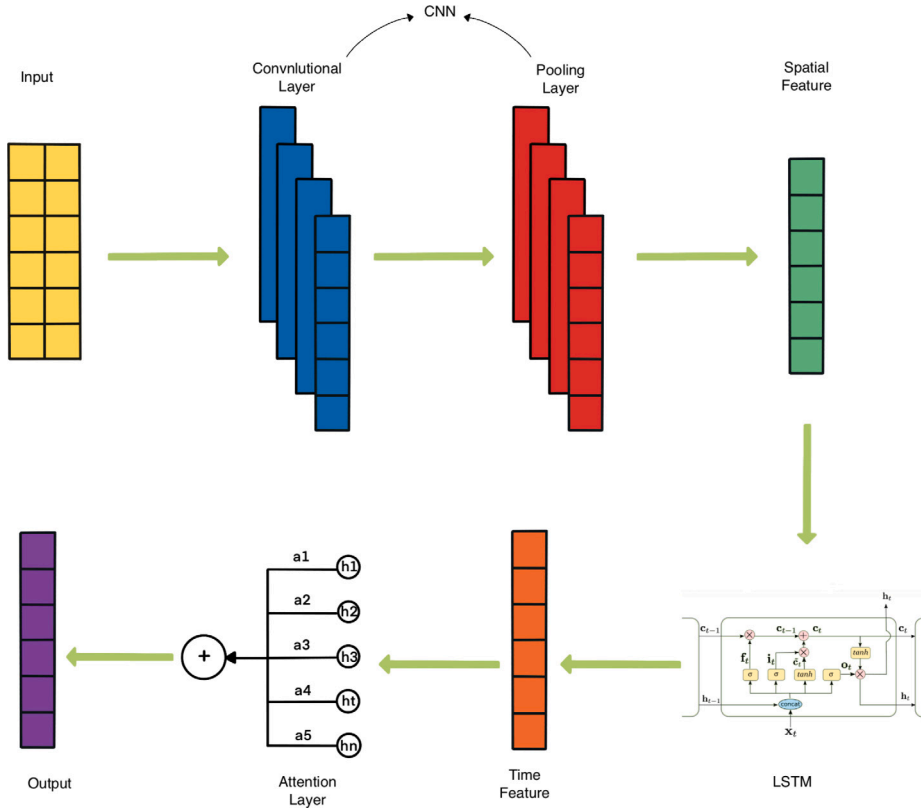


Fig. 3. The structure of the CNN-LSTM model with the attention mechanism.

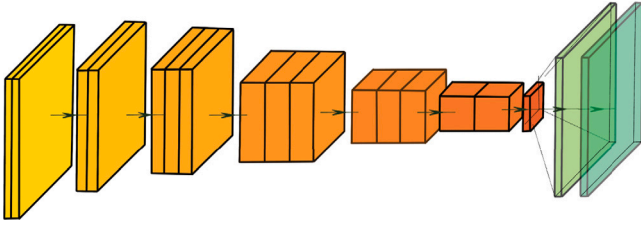


Fig. 4. Performing multiple convolution operations on the data.

By following the aforementioned steps, the target parameters can be estimated from the sample data, allowing for the derivation of the corresponding probability density function. This enables the assessment of safety levels at various locations within the region, leading to a comprehensive prediction and analysis of the environmental conditions.

3.1.2. Path optimization module

In the path optimization problem, two of the most common challenges are the multi-path problem and the multi-bend problem. The former requires a comprehensive evaluation of various paths to identify the optimal route, while the latter involves determining the most suitable bending angle and bending time. Both of these challenges typically represent multi-objective optimization problems, where there can be inherent conflicts among the objectives. As a result, path optimization demands high-performance standards. These two problems will be discussed separately in the following paragraphs.

In order to quantify the path quality of different paths, we propose different path quality evaluation functions for different problems.

In a multi-path scenario, the efficiency of a UAV swarm navigating through a path is generally influenced by several factors. These include

the path length l , the number of break points N along the path, the angles at each break point θ_x (where $x = 1, 2, 3, \dots, N$), the path feasibility ρ , the variance $D(v)$ of the average passing speed v of the UAVs, and the minimum number of feasible cross sections of the UAV, denoted as n_u , within the path feasibility region. Therefore, for a UAV swarm consisting of m' members, the quality evaluation function of the path can be expressed as shown in Eq. (16):

$$F = \omega_1 e^{-l} - \omega_2 \sum_{i=1}^N |\theta_i| - \omega_3 \rho - \omega_4 D(v) - \omega_5 \left(\frac{m'}{n_u}\right)^2 \quad (16)$$

where ρ is the probability of being in the danger zone; $\omega_j, j = 1, 2, 3, 4, 5$ are the weight parameters, which are related to the characteristics of the UAV and flight requirements and are non-negative real numbers. This multi-objective function F integrates five critical navigation aspects: path length, turning smoothness, safety, speed consistency, and swarm coordination. The weights ω_j reflect Pareto-optimal tradeoffs among these competing objectives. Therefore, the passage efficiency of different paths is analyzed according to the path quality evaluation function.

For a UAV swarm that operates as a unified entity along a designated path, path optimization involves assessing the optimization degree of each path at the current scale and selecting the one with the highest optimization degree. In the case of a decentralized UAV swarm, the journey from the starting point to the endpoint can be represented as a connected graph comprising multiple nodes. The optimal extremum F_k between each node, along with the corresponding maximum passable number n_k (where $k = 1, 2, \dots, M$, and M denotes the total number of subsets), is calculated to determine the maximum

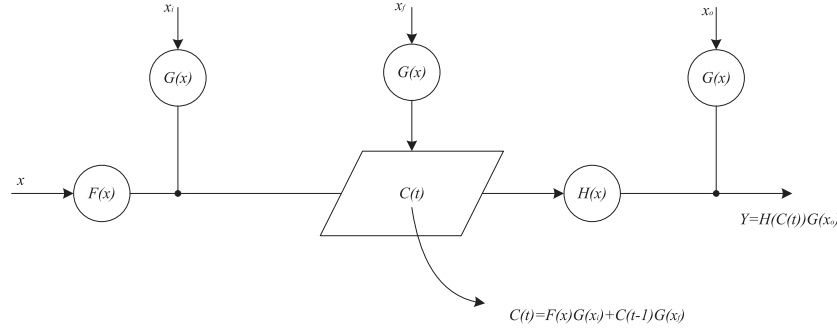


Fig. 5. The mathematical model of LSTM.

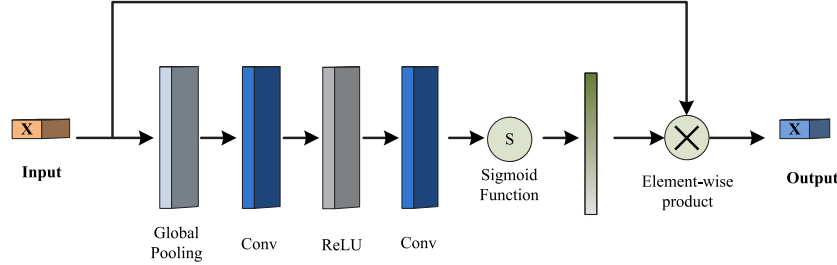


Fig. 6. The structure of the channel attention mechanism.

value of the optimization function, as illustrated in Eq. (17):

$$\begin{aligned} & \max \frac{1}{n} \sum_{k=1}^M N_k F_k \\ & \text{s.t.} \begin{cases} \sum_{k=1}^M n_k = n_u \\ M \leq n_u \\ n_k \text{ is a positive integer.} \end{cases} \end{aligned} \quad (17)$$

In the calculation process, whenever a set of $\{F_k, n_k\}$ is established, it is essential to disconnect the corresponding limit path. Subsequently, a new optimal combination $\{F_{k+1}, n_{k+1}\}$ must be recalculated until the condition $\sum_{k=1}^M n_k = n_u$ is met, or until there is no feasible path between the starting point and the endpoint. If the latter occurs, we must determine the total traffic n_{max} , after completing a round of optimal calculations. From there, we can compute the remainder $n'_u = \text{mod}(n_u, n_{max})$, ultimately allowing us to calculate the optimal planning when $n_u = n'_u$.

In a multi-bend scenario, the path of a UAV can be categorized into two segments: the turning area path and the straight area path. Typically, the straight area path does not require optimization, which means the focus is primarily on optimizing the turning area path. For a specific turning area, the goal is to develop an optimized path that outperforms the existing one. To facilitate the evaluation of the degree of path optimization, we define a quality evaluation coefficient Z for a path of length l , as expressed in Eq. (18):

$$\begin{cases} Z = F_l(\rho)(le^{-l} - \omega_1 v_0 \sin \theta_z - \omega_2 \frac{v_m^2}{r} - \omega_3 \sin \varphi) \\ F_l(\rho) < 1 \end{cases} \quad (18)$$

The function $F_l(\rho)$, which is given by the situation assessment model, represents the probability of a path with length l within the constrained domain. The weight parameters are denoted as $\omega_j, j = 1, 2, 3$. The variable v_0 indicates the initial speed at which the UAV enters the area; this speed is influenced by factors related to the UAV swarm and flight requirements, and is defined as a non-negative real

number. Meanwhile, v_m represents the maximum flight speed of the UAV. The angle θ_z denotes the angle between the tangent line at the start of the path and the initial direction of the UAV's velocity. The maximum radius of curvature for the path is represented by r , and φ is the angle between the tangent line at the end of one path section and the tangent line at the beginning of the subsequent section. By calculating the quality evaluation coefficient for the path, we can assess whether it has been optimized effectively.

Upon examining the expression, it is evident that in order for Z to reach its maximum value, the path length l , the tangent angle θ_z , and the endpoint angle φ must be minimized. Additionally, the maximum radius of curvature should be maximized. Clearly, achieving an included angle of 0° can be accomplished relatively easily. However, optimizing the curvature radius and path length is more complex, as they are inherently tied to the specific shape of the curve. To address this challenge, a cyclotron curve has been introduced to approximate the curve. This type of curve features a curvature that is proportional to the distance traversed along it, and its equation can be expressed as Eq. (19):

$$kL = A^2 \quad (19)$$

where k represent the curvature, defined as $k = 1/r$, where r is the radius of curvature. The length of the curve is denoted as L , and "A" is a constant. This cyclotron curve relation $kL = A^2$ links curvature and path length for turning optimization. It enables analytical derivation of optimal bend geometry, balancing path shortness and navigability. Consequently, the length of the curve can be related to the radius of curvature, allowing the quality evaluation coefficient to be streamlined into Eq. (20):

$$Z = le^{-l} - \omega_2 \frac{v_m^2}{A^2} l \quad (20)$$

Taking the derivative of both sides of the equation with respect to l , and setting the derivative equal to 0, yields Eq. (21):

$$Z' = (1-l)e^{-l} - \omega_2 \frac{v_m^2}{A^2} = 0 \quad (21)$$

According to this equation, we can determine the value of l . If l does not have a solution, it indicates that the optimal path is the

one with the largest angle in the turning region. Conversely, if l has a solution, the function $f(l) = (1 - l)e^{-l}$ is strictly monotone, and the solution corresponds to the maximum point. Thus, the cyclotron curve associated with l represents the desired optimal path. From this analysis, each turning region can be optimized individually to derive the final optimization path.

Combining the above two kinds of optimization methods, the path optimization task of different scenarios can be completed.

3.2. The DDPG algorithm

The DDPG algorithm is an actor-critic method designed for continuous state and action spaces. When compared to similar reinforcement learning algorithms, DDPG exhibits lower sample requirements and demonstrates superior performance in scenarios with limited data, when contrasted with the PPO algorithm. Although the DDPG algorithm has less exploration capability than the SAC algorithm, it converges more rapidly, making it particularly suitable for situations where timeliness is prioritized over precision. The following sections will delve into the algorithm from four key perspectives: the actor, the critic, the reward function, and the experience replay mechanism.

3.2.1. Actor

The actor is utilized to define the policy function, which primarily focuses on generating the output action a_t based on the current state s_t . This involves both the target policy network and the online policy network, which are updated alternately. Initially, the parameters θ^u and $\theta^{u'}$ are set to be equal. The update process for the parameters of the online policy network is illustrated in Eq. (22):

$$\nabla_{\theta^u} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q) \nabla_{\theta^u} u(s | \theta^u) |_{s=s_i, a=u(s_i)} \quad (22)$$

where N is the number of single training samples and Q is the function of state action value; θ^Q is the online Q network parameter; θ^u is the online policy network parameter. The target policy network is updated at regular intervals, as shown in Eq. (23):

$$\theta^{u'} = \alpha \theta^u + (1 - \alpha) \theta^{u'} \quad (23)$$

where α is the learning rate.

3.2.2. Critic

The critic is employed to approximate the state-action value function, which encompasses both the target Q network and the online Q network that are updated alternately. Initially, the parameters θ^Q and $\theta^{Q'}$ are set to be equal. Once experience samples (s_i, a_i, r_i, s_{i+1}) are obtained from the experience buffer, the online Q network is updated by minimizing the loss value L_c . The method for calculating L_c is detailed in Eq. (24):

$$L_c = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2 \quad (24)$$

The calculation method of y_i is shown in Eq. (25):

$$y_i = r_i + \gamma Q'(s_{i+1}, u'(s_{i+1} | \theta^{u'}) | \theta^{Q'}) \quad (25)$$

Among them, r_i is the single step reward value; γ is the discount factor; Q' is the target state action value function; u' is the target strategy function; $\theta^{u'}$ is the network parameter of the target strategy; $\theta^{Q'}$ is the target Q network parameter. Similarly, the target Q network is updated every once in a while, as shown in Eq. (26):

$$\theta^{Q'} = \alpha \theta^Q + (1 - \alpha) \theta^{Q'} \quad (26)$$

The actor network and the critic network alternate and iterate, thereby gradually improving the decision-making performance of the model.

3.2.3. Reward function

The reward function is a crucial component in reinforcement learning tasks, as it guides the updating of network parameters for both actors and critics. However, designing an effective reward function poses significant challenges. A well-structured reward function can enhance model performance, while a poorly conceived one may result in subpar outcomes or even prevent the model from converging, ultimately wasting computational resources. This challenge is particularly evident in multi-objective optimization problems, where it is essential not only to create an appropriate reward function for each objective but also to assess the relative importance of each objective in the overall framework.

To create an appropriate reward function for a multi-objective optimization problem, it is typically effective to first establish a reward function for a single objective. Following this, a weighted integration of the single objective reward function can be used to develop the comprehensive reward function. In this process, the weight values often require determination through experimentation and analysis. The overall structure of the reward function R is represented in Eq. (27):

$$R = \sum_{i=1}^n \omega_i R_i \quad (27)$$

where n is the number of targets; ω_i are the weight parameters; R_i are the reward functions for sub-objective.

3.2.4. Experience replay mechanism

The experience replay mechanism utilizes a fixed-size memory space to store past experiences, randomly selecting a predetermined number of these experiences at a time to update the network. This approach has the effect of significantly diminishing the temporal correlation between the experiences in replay, as it blends both old and new experiences for network updates. Despite this, the mechanism operates under the premise that all experiences in the pool are equally significant, allowing for an even training of the network through sampling from the experience pool. However, this assumption does not align with the principles of human learning. In human learning, experiences that are successful or associated with substantial losses tend to provide more valuable lessons. With this perspective in mind, we modified the original uniform random replay mechanism to employ a priority replay mechanism. When an experience j is acquired, we can calculate its time difference error (TD-error) δ_j , as detailed in Eq. (28):

$$\delta_j = r(s_t, a_t) + \gamma Q'(s_{t+a}, a_{t+1}, w) - Q(s_t, a_t, w) \quad (28)$$

where $Q'(s_{t+a}, a_{t+1}, w)$ is the target action-value network parameterized by w . The TD-error δ_j quantifies prediction surprise in our prioritized replay. High-error experiences indicate greater learning potential, allowing focused training on the most informative samples to accelerate convergence. Based on this, the replay probability of every experience can be expressed as Eq. (29):

$$P_j = \frac{|\delta_j|}{\sum_k |\delta_k|} \quad (29)$$

By establishing varying priorities, it becomes possible to replay more beneficial experiences with greater frequency. This approach enhances training efficiency and accelerates the convergence process.

4. The hybrid path planning model for UAV swarms based on the SAPO-DDPG

The notations used in this section are shown in Table 6.

To address the path planning challenge for UAV swarms in dynamic multi-obstacle environments, we propose the SAPO-DDPG hybrid model. This framework integrates the DDPG algorithm as the core decision-maker, the SAPO model for situational information fusion and path refinement, and the AHF algorithm for sensor data preprocessing.

Table 6

Table of notations in Section 4.

Symbol	Description
S_t	Raw sensor data stream at time t
p_s, p_g	Start and goal positions
U	UAV swarm: $U = \{u_1, u_2, \dots, u_N\}$
P^*	Optimized paths: $P^* = \{P_1^*, P_2^*, \dots, P_N^*\}$
R_1	Safety reward function
R_2	Motion stability reward function
R_3	Traffic efficiency reward function
R_4	Swarm coordination reward function
D_i	Shortest distance between UAV i and danger area
D_s	Safety distance threshold: $D_s = \frac{v_{\max}^2}{2a} + D_c$
D_c	Collision threshold (based on sensor accuracy)
$ a_t $	Acceleration magnitude at time t
θ_{lat}	Angle between velocity and acceleration vectors
T	Time spent in the mission area
L	Path length
$\mathbf{w}$	Gaussian noise vector: $\mathbf{w} = [w_x, w_y, w_\theta]^T \sim \mathcal{N}(0, \Sigma_d)$
Σ_d	Noise covariance matrix: $\text{diag}(0.1^2, 0.1^2, 0.05^2)$
τ	Yaw response time constant (0.5s)
$\mathbf{F}_{\text{wind}}$	Wind disturbance force
$\mathbf{w}(t)$	Wind velocity vector
$\mathbf{w}_b$	Base wind component
$\mathbf{w}_{g,k}$	Wind gust components
$N_{g,k}(t)$	Poisson process for gust frequency ($\lambda = 0.1$ Hz)
η_i	Obstacle intent uncertainty noise
$\mathbf{J}$	Obstacle inertia tensor
σ_o	Obstacle intent uncertainty parameter (1.5 m/s ²)
$V^{(k)}$	Lyapunov function at iteration k
F^*	Optimal path evaluation value
$\Delta V^{(k)}$	Difference in Lyapunov function
α, β	Convergence rate coefficients
η	Optimization step size
L	Lipschitz constant of ∇F
$\epsilon_{\max}$	Upper bound of environmental uncertainty

The proposed hybrid model operates through four synchronized phases:

(1) *Real-time Perception*: Raw sensor data undergoes adaptive noise filtering via AHF, where dynamic parameter updates (K , N_e , f_n) are computed using iterative loss minimization.

(2) *Situation Assessment*: Filtered data feeds into the channel-attention-enhanced CNN-LSTM for spatiotemporal feature extraction. The output constructs a Gaussian danger field whose parameters (μ , σ) are estimated via maximum likelihood.

(3) *Policy Generation*: The DDPG agent receives the danger probability map and feasible domain constraints. Its actor network outputs continuous actions using prioritized experience replay, while the critic evaluates multi-objective rewards.

(4) *Path Refinement*: Raw trajectories from DDPG are optimized using SAPO's breakpoint-curvature tradeoff and multi-path evaluation. This closed-loop process repeats at 10 Hz.

The structure of this model is illustrated in Fig. 7.

The execution pipeline is illustrated in Fig. 7 and detailed in Algorithm 1.

As illustrated in Fig. 7, when the input is received, the data undergoes initial preprocessing to perform denoising and smoothing operations using the AHF algorithm. Subsequently, the processed data is input into the CNN and LSTM for the extraction of spatial and temporal features, with a channel attention mechanism employed to enhance the focus on significant features. Following the feature extraction and analysis, the situation assessment model constructs a probability density function based on the results and utilizes existing data for parameter estimation, leading to the probability distribution that indicates whether each point in the environment is safe and feasible. This probability distribution enables the determination of the feasible domain for its own state and actions, after which the DDPG algorithm can be applied to train the strategy model. Once the strategy model is developed, optimal decision-making can be swiftly achieved via the state. To

Algorithm 1 SAPO-DDPG Hybrid Path Planning Framework

Require: Sensor stream S_t , Start position p_s , Goal position p_g , UAV swarm $U = \{u_1, u_2, \dots, u_N\}$

Ensure: Optimized collision-free paths $P^* = \{P_1^*, P_2^*, \dots, P_N^*\}$ for all UAVs

- 1: **Initialize:**
- 2: Actor network $\pi(s|\theta^\pi)$, Critic network $Q(s, a|\theta^Q)$, Target networks $\pi'(s|\theta^{\pi'})$, $Q'(s, a|\theta^{Q'})$ with $\theta^{\pi'} \leftarrow \theta^\pi$, $\theta^{Q'} \leftarrow \theta^Q$, Replay buffer B with capacity C , SAPO modules (AHF filter, CNN-LSTM-Attention network), Situation assessment parameters Θ_{SAPO}
- 3: **for** episode = 1 **to** M **do**
- 4: Reset environment, receive initial state s_0 for all UAVs $u \in U$
- 5: **for** $t = 1$ **to** T **do**
- 6: **Phase 1: Real-time Perception**
- 7: **for** each UAV $u \in U$ **do**
- 8: $\tilde{S}_t^u \leftarrow \text{AHF}(S_t^u)$ {Apply Adaptive HANTs Filter}
- 9: **end for**
- 10: **Phase 2: Situation Assessment**
- 11: $F_t \leftarrow \text{CNN-LSTM-Attention}(\{\tilde{S}_t^u | u \in U\})$
- 12: Estimate Gaussian parameters: $\hat{\mu}, \hat{\sigma}^2 \leftarrow \text{MLE}(F_t)$
- 13: Construct danger probability map $\mathcal{D}$:
- 14: **for** each grid cell (i, j) in environment **do**
- 15: $d_{ij} \leftarrow \min(\text{Euclidean distance to all obstacle centroids})$
- 16: $D_{ij} \leftarrow \frac{1}{\sqrt{2\pi\hat{\sigma}^2}} \exp\left(-\frac{(d_{ij}-\hat{\mu})^2}{2\hat{\sigma}^2}\right)$ {Gaussian danger field}
- 17: **end for**
- 18: **Phase 3: Policy Generation & Execution**
- 19: **for** each UAV $u \in U$ **do**
- 20: Observe current state $s_t^u = [\text{pose}^u, \mathcal{D}, \text{swarm_states}]$
- 21: Select action: $a_t^u \leftarrow \pi(s_t^u|\theta^\pi) + \mathcal{N}_t$ {Add exploration noise}
- 22: Execute action a_t^u , observe reward r_t^u and next state s_{t+1}^u
- 23: Store transition $(s_t^u, a_t^u, r_t^u, s_{t+1}^u)$ in replay buffer B
- 24: **end for**
- 25: **Phase 4: Path Refinement**
- 26: $P_{\text{raw}} \leftarrow \text{Trajectory from executed actions } \{a_t^u | u \in U\}$
- 27: $P^* \leftarrow \text{SAPO-Optimize}(P_{\text{raw}})$ {Curvature-breakpoint optimization}
- 28: **Model Learning (Prioritized Experience Replay)**
- 29: **if** $|B| > \text{batch_size}$ **then**
- 30: Sample mini-batch of K transitions from B with probability $P(j) = \frac{|\delta_j|}{\sum_k |\delta_k|}$
- 31: **for** each sampled transition j **do**
- 32: Compute target value: $y_j = r_j + \gamma Q'(s_{j+1}, \pi'(s_{j+1}|\theta^{\pi'}))|\theta^{Q'}$
- 33: Compute TD-error: $\delta_j = y_j - Q(s_j, a_j|\theta^Q)$
- 34: Update priority: $p_j \leftarrow |\delta_j| + \epsilon$
- 35: **end for**
- 36: Update critic by minimizing: $L = \frac{1}{K} \sum_j (y_j - Q(s_j, a_j|\theta^Q))^2$
- 37: Update actor using policy gradient: $\nabla_{\theta^\pi} J \approx \frac{1}{K} \sum_j \nabla_a Q(s, a|\theta^Q)|_{s=s_j, a=\pi(s_j)} \nabla_{\theta^\pi} \pi(s|\theta^\pi)|_{s_j}$
- 38: Update target networks: $\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$, $\theta^{\pi'} \leftarrow \tau\theta^\pi + (1-\tau)\theta^{\pi'}$
- 39: **end if**
- 40: **end for**
- 41: **end for**
- 42: **return** P^*

comprehensively address the needs of various targets and mitigate the risk of reinforcement learning algorithms becoming trapped in local optima, the AHF algorithm is again employed to smooth the sequence of decision actions, followed by a path optimization method to finalize the optimization of details.

To establish the model, it is crucial to determine the appropriate weighting coefficients for the reward function in the DDPG model, as well as for the path quality evaluation function in the path optimization model, in relation to the specific objectives of the problem at hand. Meanwhile, the uncertainty modeling and management methods are presented at the end of this section.

4.1. Design of reward function

The reward function for the DDPG agent is designed to balance four critical objectives: safety R_1 , motion stability R_2 , traffic efficiency

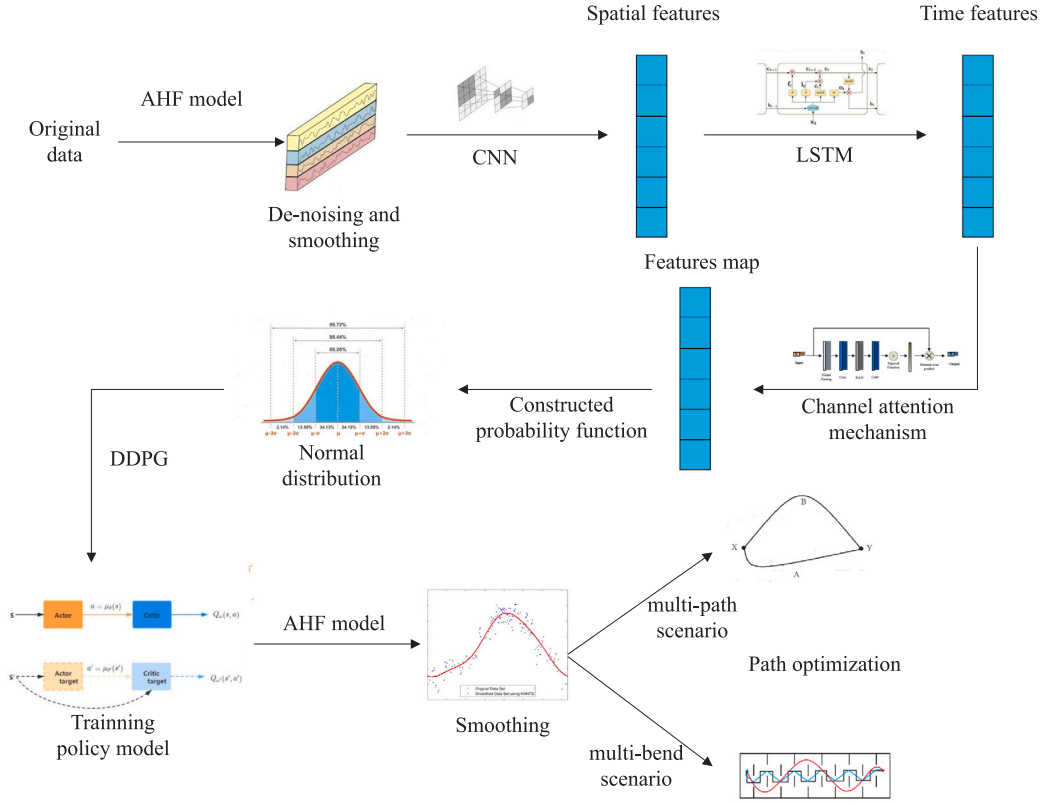


Fig. 7. The SAPO-DDPG execution pipeline: (a) Input preprocessing via AHF, (b) Spatiotemporal encoding through attention-weighted CNN-LSTM, (c) DDPG decision-making within SAPO-derived action constraints, (d) Path smoothing via curvature optimization.

R_3 , and swarm coordination R_4 . To achieve this, a distinct reward function is designed for each of these four goals. These sub-objective reward functions are then weighted and combined to form an integrated approach.

The multi-objective reward function integrates four components, each addressing a critical navigation requirement:

Safety: Penalizes proximity to obstacles based on distance thresholds.

Motion Stability: Rewards smooth acceleration and directional alignment.

Traffic Efficiency: Encourages timely path completion.

Swarm Coordination: Prevents inter-agent collisions.

The safety reward (Eq. (30)) penalizes actions where the minimum distance $\min_i\{D_i\}$ between any UAV and a danger area is less than the safety threshold D_s . The penalty is inversely proportional to the distance D_i , $i = 1, 2, \dots, n$, imposing a higher cost for closer proximity to obstacles. The safety distance D_s (Eq. (31)) is derived from the UAV's kinematics and sensor accuracy.

$$R_1 = \begin{cases} -\sum_{i=1}^n \frac{D_s}{D_i} & \text{if } \min_i\{D_i\} \leq D_s \\ 0 & \text{if } \min_i\{D_i\} > D_s \end{cases} \quad (30)$$

where n is the number of the UAVs in the swarm.

$$D_s = \frac{v_{\max}^2}{2a} + D_c \quad (31)$$

where D_c is the collision threshold, which is determined by twice the on-board RTK-GPS positioning error.

Motion stability encompasses two primary aspects. First, the rate of change in speed should not be excessive, meaning that acceleration should be minimized as much as possible. Second, the trajectory of movement should be smooth to prevent sharp turns from adversely affecting the stable posture of the body of the UAV; this entails keeping

the angle between speed and acceleration as small as possible. By integrating these two considerations, we can conclude that a smaller absolute value of acceleration leads to a higher reward value, while a smaller angle between acceleration and velocity also results in a greater reward value. Therefore, the reward function R_2 for motion stability can be defined as outlined in Eq. (32):

$$R_2 = 3 - |a_t| + \frac{1}{\cos \theta_{vat}} \quad (32)$$

where $|a_t|$ is the acceleration at the moment t ; θ_{vat} is the angle between speed and acceleration at the moment t .

Traffic efficiency primarily focuses on measuring the time it takes for an entire UAV swarm to traverse a specific area. Consequently, the reward function for traffic efficiency should be a dynamic, time-dependent function. As time progresses, this function progressively increases the penalty value to encourage the model to reach its target as swiftly as possible. Therefore, the reward function R_3 for traffic efficiency can be represented as outlined in Eq. (33):

$$R_3 = \begin{cases} 0 & \text{if } T \leq \frac{2L}{v_{\max}} \\ -\frac{D_e}{20v_{\max}} & \text{if } T > \frac{2L}{v_{\max}} \end{cases} \quad (33)$$

where T is the time in the area; L is the path length; $v_{\max}$ is the maximum speed of the UAV; D_e is distance from the exit. Considering that it is impossible for the UAV to keep a constant maximum speed and fly in a straight line, a 100% margin is given.

Finally, the purpose of the internal order of the UAV swarm is to avoid internal collisions, so the reward function R_4 is shown as Eq. (34):

$$R_4 = \begin{cases} -9999 & \text{if UAVs collide} \\ 0 & \text{if UAVs do not collide} \end{cases} \quad (34)$$

These reward components address core navigation requirements: R_1 ensures safety through distance penalties, R_2 promotes motion stability, R_3 encourages timely completion, and R_4 prevents inter-agent collisions. Their weighted combination guides multi-objective optimization.

Rewards are computed at 100 Hz intervals:

- Safety reward R_1 (Eq. (30)) uses Euclidean distance D_i from UAV to nearest obstacle cluster centroid (updated via onboard LiDAR).
- Motion stability R_2 (Eq. (32)) derives θ_{uat} from IMU-based pose estimation.
- Traffic efficiency R_3 (Eq. (33)) starts timing (T) when the lead UAV enters a sector.
- Collision checks R_4 (Eq. (34)) use axis-aligned bounding boxes with $\pm D_c$ tolerance.

Upon establishing the reward function for the subitem, it is essential to ascertain the appropriate weight for each reward function. In the forthcoming experiment, various weights will be assigned to identify the optimal weighting configuration.

4.2. Determination of weighting coefficient

In the path optimization model, we define a path quality evaluation function, as presented in Eqs. (16) and (18), to differentiate the optimization levels of various paths. However, the weights in these functions must be tailored to meet the specific requirements of the problem at hand. In the context of UAV swarm path planning within a dynamic multi-obstacle environment, when confronted with the multi-path issue, our focus primarily lies on the safety and traffic efficiency of the paths, while we place less emphasis on traffic flow limits and the number of turns. Consequently, we can assign zero values to weights ω_2 and ω_5 . Additionally, given that safety takes precedence over traffic efficiency, and that the variance in path length serves as a better indicator of traffic efficiency than speed, we set the weights as follows: $\omega_1 = 0.3$, $\omega_2 = 0$, $\omega_3 = 0.5$, $\omega_4 = 0.1$, $\omega_5 = 0$. The resulting path quality function is illustrated in Eq. (35):

$$F = 0.3e^{-l} - 0.5\rho - 0.1D(v) \quad (35)$$

The theoretical basis for weight selection is given below:

The weight parameters ω_j in Eq. (16) embody Pareto-optimal trade-offs among conflicting objectives:

- *Safety priority*: $\omega_3 > \omega_1$ (path risk ρ dominates path length l)
- *Swarm coordination*: ω_5 penalizes bottleneck formation (m'/n_u)

This aligns with UAV kinematic constraints and swarm dynamics. The specific values in Eq. (35) derive from experimental Pareto-front analysis.

In the context of a multi-bend scenario, centripetal acceleration serves as an indicator of both the smoothness of the path and the quality of traffic flow. The initial and final angles, representing the starting and ending points of the reaction path, are not only disregarded in the calculation of the extreme value but also have minimal impact on the overall optimization of the path. Accordingly, the following parameters have been established: $\omega_1 = \omega_3 = 0.01$ and $\omega_2 = 0.98$. The final function is represented in Eq. (36):

$$\begin{cases} Z = F_l(\rho)(le^{-l} - 0.01v_0 \sin \theta_z - 0.98 \frac{v_m^2}{r} - 0.01 \sin \varphi) \\ F_l(\rho) < 1 \end{cases} \quad (36)$$

After simplification, the above objective function can be more easily differentiated and extreme points can be found, thereby obtaining an optimized scheme.

Table 7

Sensor uncertainty parameters and compensation strategies.

Sensor	Latency (ms)	Noise (σ)	Compensation method
LiDAR	50	0.25 m	AHF temporal smoothing
IMU	20	0.1 m/s ²	Kalman filtering
GPS	100	0.5 m	Dead reckoning fusion

4.3. Uncertainty modeling and management

The SAPO-DDPG framework explicitly addresses four sources of uncertainty.

4.3.1. Dubins vehicle dynamics with noise

The UAV kinematics follow modified Dubins model with Gaussian noise as shown in Section 4.3.1.

$$\begin{cases} \dot{x} = v \cos \theta + w_x \\ \dot{y} = v \sin \theta + w_y \\ \dot{\theta} = \frac{v}{\tau} + w_\theta \end{cases} \quad \text{where } \mathbf{w} = [w_x, w_y, w_\theta]^T \sim \mathcal{N}(0, \Sigma_d) \quad (37)$$

Here $\tau = 0.5s$ represents the yaw response time constant, and $\Sigma_d = \text{diag}(0.1^2, 0.1^2, 0.05^2)$ captures translational (x, y) and rotational (θ) disturbance covariance. This modified Dubins model incorporates Gaussian noise to account for real-world UAV dynamics uncertainties. The covariance matrix Σ_d captures different noise characteristics in translational and rotational dimensions. This models real-world control surface imperfections and atmospheric micro-turbulence.

4.3.2. Wind gust disturbances

Wind effects combine steady flow and stochastic gusts as shown in Eq. (38).

$$\mathbf{F}_{\text{wind}} = \frac{1}{2} \rho C_d A \|\mathbf{v}_{\text{uav}} - \mathbf{w}(t)\|(\mathbf{w}(t) - \mathbf{v}_{\text{uav}}) \quad (38)$$

where $\mathbf{w}(t) = \mathbf{w}_b + \sum_{k=1}^{N_g(t)} \mathbf{w}_{g,k} \cdot \mathbb{I}_{[t_k, t_k + \Delta t_k]}$. The base wind $\mathbf{w}_b \sim \mathcal{U}(0, 5)$ m/s is uniformly distributed, while gusts follow Poisson process $N_g(t) \sim \text{Poisson}(\lambda = 0.1\text{Hz})$ with exponential magnitude $\|\mathbf{w}_{g,k}\| \sim \text{Exp}(0.2)$.

4.3.3. Sensor imperfections

Table 7 quantifies the key sensor limitations:

4.3.4. Obstacle motion uncertainty

Dynamic obstacles follow stochastic dynamics as shown in Eq. (39).

$$\mathbf{v}_{\text{obs}}(t + \Delta t) = \mathbf{v}_{\text{obs}}(t) + \underbrace{\boldsymbol{\eta}_t}_{\text{intent noise}} + \underbrace{\mathbf{J}\mathbf{w}_t}_{\text{wind effect}} \quad (39)$$

where $\boldsymbol{\eta}_t \sim \mathcal{N}(0, \sigma_o^2 \mathbf{I})$ models intent uncertainty ($\sigma_o = 1.5$ m/s²), and $\mathbf{J}$ is the obstacle's inertia tensor. The CNN-LSTM predictor learns to estimate $\mathbb{E}[\boldsymbol{\eta}_t]$ from historical observations.

4.4. Convergence proofs

To establish the theoretical robustness of SAPO-DDPG, we provide convergence analysis for both the SAPO optimization and the DDPG training process.

Convergence of SAPO Path Optimization.

Under Lipschitz-continuous environmental dynamics (Khalil, 2002), the SAPO path evaluation function F (Eq. (15)) converges to a local optimum. Consider the Lyapunov function: (Eq. (40))

$$V^{(k)} = F^{(k)} - F^* \quad (40)$$

where F^* is the optimal value. The difference $\Delta V^{(k)} = V^{(k+1)} - V^{(k)}$ satisfies (Eq. (41)):

$$\Delta V^{(k)} \leq -\alpha \|\nabla F^{(k)}\|^2 + \beta \epsilon_{\max} \quad (41)$$

Table 8
Table of notations in Section 5 with values.

Parameter	Description	Subsection	Value
K	Error threshold for filtering	5.2, 5.4–5.5	= 10
N_e	Maximum deletion points	5.2, 5.4–5.5	= 20
f_n	Low-frequency component count	5.2, 5.4–5.5	= 9
D_s	Safety distance threshold	5.1,5.3–5.5	≈ 10 m
D_c	Collision threshold	5.1,5.3–5.5	= 0.5 m
$v_{\max}$	Maximum UAV flight speed	5.1,5.3–5.5	= 15 m/s
a	Acceleration range	5.1,5.3–5.5	$\in [-12, 12]$ m/s ²
T_s	Sampling rate	5.1,5.3–5.5	= 0.1 s ⁻¹
N	Number of UAVs in swarm	5.1,5.3–5.5	= 5
L_{sl}	Straight-line distance	5.1,5.3–5.5	= $\sqrt{34} \times 10^2$ m
α	Learning rate of DDPG	5.1,5.3–5.5	= 0.99
γ	Discount factor of DDPG	5.1,5.3–5.5	= 0.8
τ	Yaw response time constant	5.5.3	= 1.0 s
λ	Wind gust Poisson frequency	5.5.3	= 0.1 Hz
σ_o	Obstacle intent uncertainty	5.5.3	= 3.0 m/s ²
Network width	The size of the convolutional neural network	5.4–5.5	5-3-3-1(4-layers)
Seed	Random seed	5.1,5.3–5.5	42-51(10 trials)

where: $\alpha = \eta(2 - \eta L) > 0$ is the convergence rate coefficient, with η as the optimization step size and L as the Lipschitz constant of ∇F . $\beta > 0$ is the uncertainty scaling factor, bounded by the maximum curvature of F . $\epsilon_{\max}$ is the upper bound of environmental uncertainty (wind, sensor noise, etc.). For $\epsilon_{\max} < \alpha \|\nabla F\|^2 / \beta$, $\Delta V^{(k)} < 0$ guarantees monotonic convergence.

Variance Reduction in DDPG.

Prioritized experience replay (Eq. (28)) reduces policy gradient variance by $\mathcal{O}(1/\sqrt{N})$ (Schaul et al., 2016) versus uniform sampling. The TD-error prioritization minimizes Kullback–Leibler divergence between optimal and sampled distributions, tightening the bound in policy gradient updates as Eq. (42):

$$\text{Var}(\nabla_{\theta} J)_{\text{Prioritized}} \leq \frac{\text{Var}(\nabla_{\theta} J)_{\text{Uniform}}}{\sqrt{N}} \quad (42)$$

These convergence proofs establish theoretical foundations: Eqs. (40)–(41) guarantee SAPO optimization convergence under environmental uncertainty, while Equation (42) demonstrates prioritized replay's variance reduction for stable DDPG training.

5. Experiment and analysis

To assess the effectiveness of the algorithm, this paper undertakes a series of experiments as follows: Experiment 1 involves varying weight coefficients to identify the optimal combination for the reward function. In Experiment 2, the AHF model is compared to the original model to determine whether improvements in filtering and smoothing effects are evident. Experiment 3 evaluates the enhanced DDPG model against the original model to examine potential gains in training efficiency and decision-making performance. Experiment 4 contrasts the mixed path planning model with the current advanced path planning model to analyze the performance of each test model across different environments. Lastly, Experiment 5 conducted supplementary experiments, including ablation experiments, scalability verification, and extreme condition testing, to comprehensively examine the overall performance of the model. The experimental summary will be provided at the end of the experimental section.

All the important experimental parameters presented in Section 5, along with their application scopes and values, are shown in Table 8.

5.1. Reward function weight value selection

The experiment was conducted with the following parameters: maximum flight speed $v_{\max} = 15$ m/s, acceleration range $a \in [-12, 12]$ m/s², collision threshold $D_c = 0.5$ m, safety distance $D_s \approx 10$ m, swarm size $N = 5$, sampling rate $T_s = 0.1$ s⁻¹, straight-line distance $L_{sl} = \sqrt{34} \times 10^2$ m, DDPG learning rate $\alpha = 0.99$, and discount factor $\gamma = 0.8$.

The training and test scenes (Figs. 8 and 9) consisted of rectangular areas (500 m $\times$ 300 m). All models were trained in the training scene and evaluated in the test scene. To ensure comparability, we employed an early termination criterion: if the total path length exceeded 8000 m, the planning was terminated. We conducted 10 Monte Carlo runs per weight configuration and evaluated four key metrics: path length L , completion time T , average absolute acceleration $|\bar{a}|$, and minimum obstacle distance $d_{\min}$.

The experimental results are shown in Table 9.

Table 9 presents the experimental results. The weight ratio $\omega_1 : \omega_2 : \omega_3 : \omega_4 = 1 : 15 : 20 : 1$ achieved the best performance, yielding the shortest path length (1619.5 ± 6.8 m) and near-optimal other metrics. The final reward function is defined as Eq. (43):

$$R = R_1 + 15R_2 + 20R_3 + R_4 \quad (43)$$

Note that this weight combination is specific to our experimental scenarios and should be recalibrated for different environments or task requirements.

5.2. Comparative experiment of filtering algorithms

This experiment evaluates the filtering performance of our proposed AHF method against the standard HANTs approach. We used acceleration time-series data collected during a 5-minute UAV flight.

The AHF model was initialized with $K_0 = 10$, $N_{e0} = 20$, and $f_0 = 9$, while the HANTs parameters were set to $K = 2$, $N_e = 10$, and $f = 3$. We compared the number of iterations (N_I), time mean square deviation ($TMSD$), and smoothing rate (SR), calculated as Eq. (44)–(45):

$$TMSD = \frac{\sum_{k=1}^N (x_{ok} - x_{ik})^2}{N} \quad (44)$$

$$SR = \frac{A_i - A_o}{A_i} \quad (45)$$

where x_{ik} is the k_{th} data before filtering, x_{ok} is the k_{th} data after filtering, A_i is the amplitude of the data before filtering, and A_o is the amplitude of the data after filtering.

Figs. 10–12 illustrate the distribution of the three groups of original data alongside the waveforms that have been filtered using the two methodologies. Additionally, Table 10 presents a comparative analysis of the performance of the two methods across these three distinct datasets.

According to the data presented in Figs. 10–12, we can see that the AHF model demonstrates superior performance in filtering and smoothing effects among the three groups of experimental data. It is clear that the AHF model more accurately reflects the distribution characteristics and trends of the data when compared to the HANTs model.

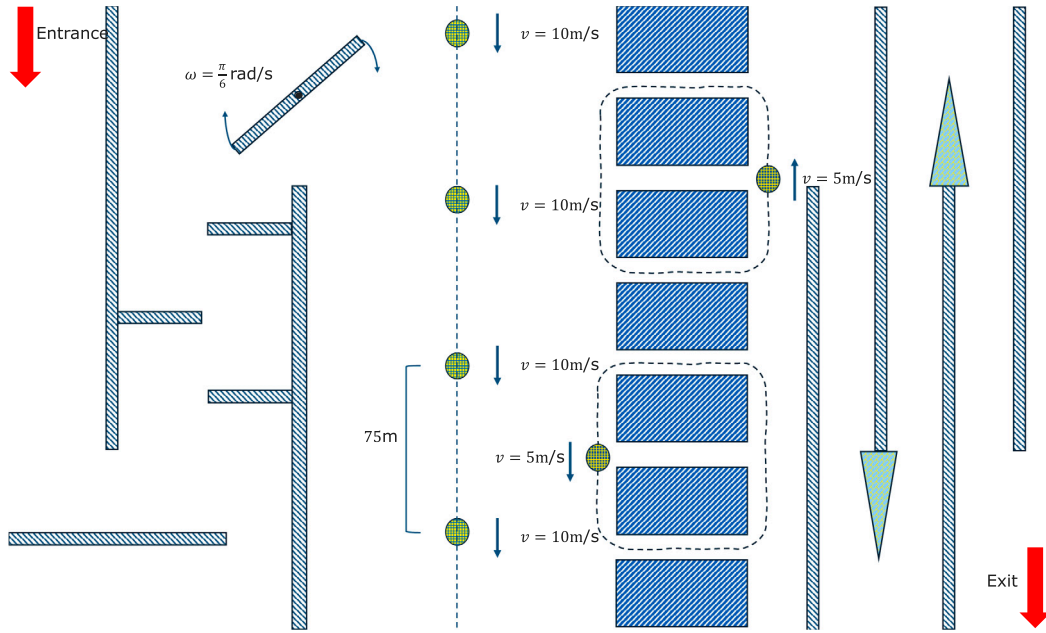


Fig. 8. The setting of the training scene.

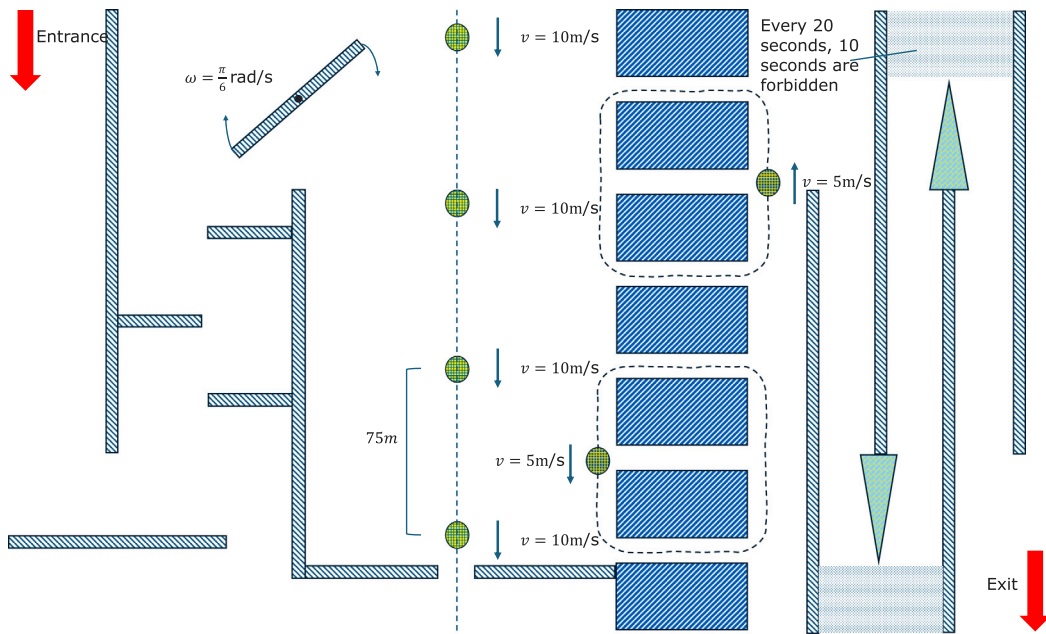


Fig. 9. The setting of the test scene.

Table 9

The path planning performance with different weight values (mean $\pm$ std).

$\omega_1 : \omega_2 : \omega_3 : \omega_4$	$L(m)$	$T(s)$	$ \ddot{a} (m/s^2)$	$d_{\min}(m)$
1:1:1:1	> 8000	—	—	—
1:5:5:1	> 8000	—	—	—
1:10:10:1	5816.1 ± 23.6	721.8 ± 12.9	1.08 ± 0.11	10.04 ± 0.07
1:15:15:1	1829.7 ± 9.7	296.8 ± 7.8	0.85 ± 0.05	10.15 ± 0.05
1:20:20:1	> 8000	—	—	—
1:20:15:1	> 8000	—	—	—
1:15:20:1	1619.5 ± 6.8	282.4 ± 5.9	0.88 ± 0.06	10.13 ± 0.03
1:15:25:1	> 8000	—	—	—
1:15:19:1	1650.9 ± 7.2	281.9 ± 5.7	0.88 ± 0.06	10.12 ± 0.03
1:15:21:1	1688.7 ± 7.1	307.7 ± 6.1	0.92 ± 0.06	10.13 ± 0.03

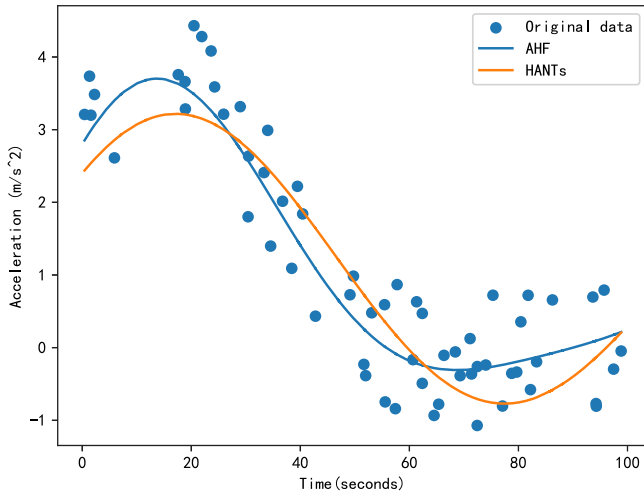


Fig. 10. The first set of original data and its filtered waveform.

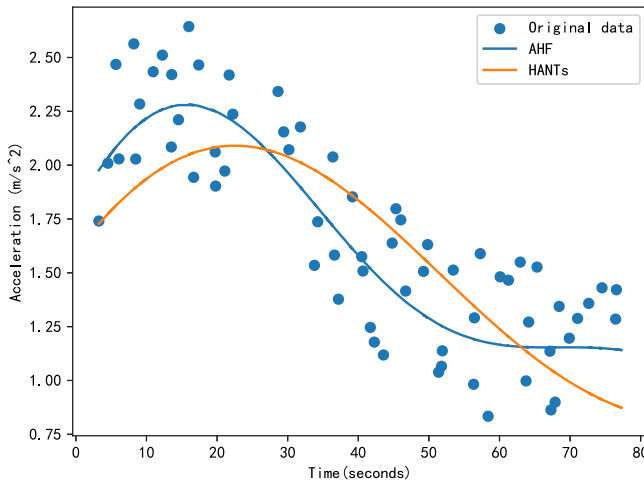


Fig. 11. The second set of original data and its filtered waveform.

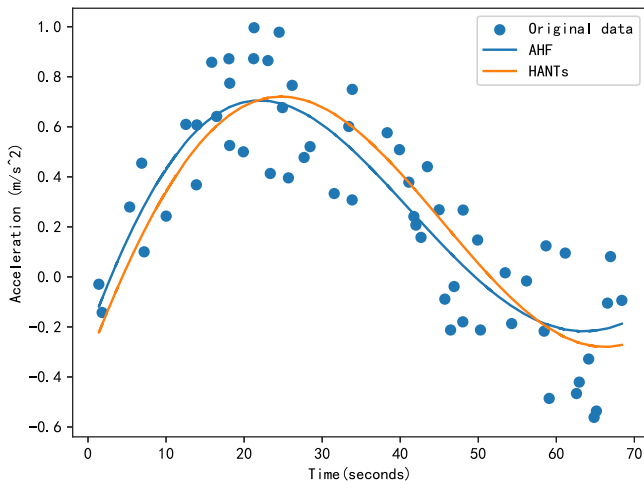


Fig. 12. The third set of original data and its filtered waveform.

As indicated in Table 10, in the first data set, the smoothing rate SR of both models is nearly identical. However, the AHF model exhibits a significant advantage in terms of time mean square deviation $TMSD$. In the subsequent two sets of data, the AHF model continues to show stronger performance in both SR and $TMSD$ metrics. This enhanced performance can be attributed to the AHF model's ability to adjust

Table 10

The performance comparison and analysis of two methods on three sets of data.

Group number	N	AHF model			HANTs model		
		N_I	$TMSD$	SR	N_I	$TMSD$	SR
1	62	56	0.322	27.1%	42	0.506	27.5%
2	65	64	0.062	37.0%	53	0.129	32.7%
3	59	49	0.037	40.8%	43	0.041	35.8%

the number of frequencies fitted to the waveform, selecting the subset of results with the least loss from the original data for output. This approach mitigates the impact of artificially set parameters on model performance. Nevertheless, due to the requirement of selecting the optimal results from the best options, the AHF model does not excel in the number of iterations N_I . In comparison to the HANTs model, the AHF model necessitates a greater number of iterations to achieve convergence.

The purpose of this experiment is to verify whether the addition of an adaptive mechanism has a positive effect on the filtering performance. It can be seen from the experimental results that through improvement, the performance of the AHF model has indeed surpassed that of the HANTs model. However, this does not mean that the AHF model can completely replace the HANTs model. Compared with the AHF model, the HANTs model has a simpler structure. Moreover, for experienced users, the HANTs model theoretically has performance close to that of the AHF model under excellent initial parameters. For the dynamic and complex scenarios targeted by this research, due to the complexity and uncertainty of the original data, it is difficult to determine effective initial parameters through experience. Therefore, the AHF model is more suitable to be used as the filtering model.

5.3. Comparative experiment of DDPG model training

To evaluate the effectiveness of our enhanced DDPG algorithm, we conducted comparative experiments by training both the original algorithm model and our proposed improved model. The experimental setup is illustrated in Fig. 8, with the relevant parameter settings consistent with those used in the previous reward function weight value selection experiment. We employed the same reward function as described in Eq. (43). A total of 300 training episodes were carried out, each lasting 3 min. Throughout the experiment, we recorded the cumulative reward value for each training episode. By analyzing the trend of the cumulative reward values, we can compare the convergence rates of the two models.

As illustrated in Fig. 13, the original model reaches the optimal path after approximately 130 episodes, with the cumulative reward stabilizing around 250 episodes. In contrast, the improved model achieves the optimal path in about 100 episodes, with cumulative rewards converging around 210 episodes. This demonstrates that enhancements to the experience reply mechanism significantly accelerate the convergence speed of the DDPG model, allowing it to identify the optimal decision-making strategy more quickly and efficiently.

The significance of this experiment lies in comparing the differences in training efficiency between the DDPG models before and after improvement. It can be seen that the improved DDPG model, compared with the original model, has 23% fewer episodes approaching the reward peak and 16% fewer episodes completing convergence. This indicates that the improved DDPG model has indeed enhanced the training efficiency and has a beneficial effect on the overall training speed of the model to a certain extent.

5.4. Comparative experiment of several path planning models

To evaluate the effectiveness of the algorithm, we compared the trained SAPO-DDPG model with several established models, including APF-RRT* (Xu et al., 2023a), GA-RRT (Fan et al., 2024), B-APFDQN (Kong et al., 2023), and TSEB-DDPG (Tian et al., 2024). The testing scene is illustrated in Fig. 9.

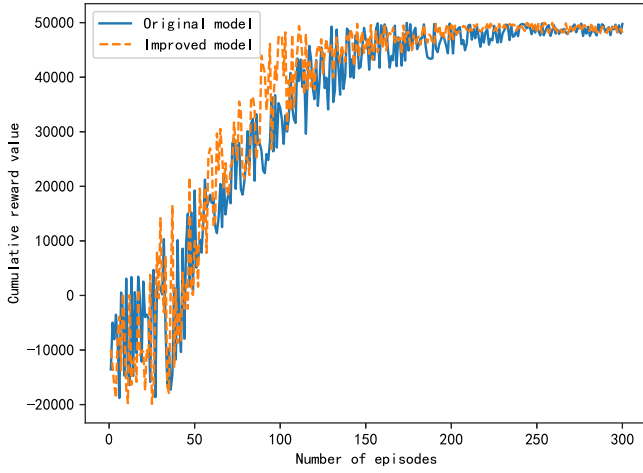


Fig. 13. The trend of total reward value during training.

Table 11

The experimental results of single UAV path planning (mean $\pm$ std).

Algorithm	T_i	$L_p(m)$	$T_c(s)$	n_c
APF-RRT*	278.4 $\pm$ 9.7 s	2461.2 $\pm$ 37.2	278.4 $\pm$ 9.7	3.3 $\pm$ 1.49
GA-RRT	203.2 $\pm$ 14.2 s	2170.4 $\pm$ 21.4	203.2 $\pm$ 14.2	1.2 $\pm$ 0.78
B-APFDQN	15.8 $\pm$ 0.2 h	2001.4 $\pm$ 18.7	137.8 $\pm$ 11.1	0.7 $\pm$ 0.82
TSEB-DDPG	23.8 $\pm$ 0.2 h	1860.3 $\pm$ 15.5	121 $\pm$ 8.7	0
SAPO-DDPG	29.4 $\pm$ 0.2 h	1821.4 $\pm$ 13.4	103 $\pm$ 8.9	0

In this experiment, a 4-layer convolutional network with a 5-3-3-1 structure is used. The convolutional kernels used in the first layer are 5×5 convolutional kernels, which are used for extracting features over a wide range. This not only effectively reduces parameters but also does not generate a large amount of computation. The subsequent two layers adopt 3×3 convolutional kernels, which can control the number of parameters and reduce the complexity of the operation at the same time. The last layer adopts a 1×1 convolutional kernel, which is equivalent to a fully connected layer.

Our experiments encompassed two main components: the path planning for a single UAV and the path planning for a UAV swarm. For the single UAV path planning, we focused on several key metrics, including training time T_i , path length L_p , completion time T_c , collision frequency n_c , and the shape of the path generated by different algorithms. In the UAV swarm path planning test, we primarily assessed the total path length of the swarm L_t , the maximum time difference in completion T_d , and the number of collisions N_c encountered.

Figs. 14–15 shows the planned paths of the five models in a certain run of experiments. Table 11 shows the data comparison of the corresponding indicators of these models.

The APF-RRT* and GA-RRT models do not incorporate a neural network structure, which means they do not require pre-training; their training time aligns with their overall completion time. As illustrated in Table 11, the iteration time for the SAPO-DDPG algorithm is longer than that of the other algorithms, primarily due to SAPO-DDPG's more complex architecture. While the convergence speed of the DDPG model is enhanced, the training time paradoxically increases because of the secondary optimization introduced by the SAPO algorithm. In terms of path length, completion time, and collision frequency, the SAPO-DDPG algorithm exhibits distinct advantages. This can be attributed to its approach of multiple optimization policies, which not only preserves the efficient decision-making capabilities of reinforcement learning but also enhances path optimization through the secondary adjustment of SAPO. It can be seen that the optimization quality of the path in the SAPO-DDPG algorithm surpasses that of other algorithms. Furthermore, it is worth noting that, from the perspective of planning trajectories, unlike

Table 12

The experimental results of UAV swarm path planning (mean $\pm$ std).

Algorithm	$L_t(m)$	$T_d(s)$	N_c
APF-RRT*	14932.5 $\pm$ 1429.6	26.4 $\pm$ 3.28	16.3 $\pm$ 5.44
GA-RRT	12315.3 $\pm$ 1017.3	21.3 $\pm$ 5.31	5.9 $\pm$ 4.31
B-APFDQN	10897.4 $\pm$ 831.72	23.2 $\pm$ 4.72	3.3 $\pm$ 4.40
TSEB-DDPG	10115.8 $\pm$ 759.38	21.1 $\pm$ 4.53	0.4 $\pm$ 0.89
SAPO-DDPG	9691.6 $\pm$ 314.65	16.2 $\pm$ 3.65	0.2 $\pm$ 0.67

other reinforcement learning models, the SAPO-DDPG model notices the differences between the test scenarios and the training scenarios and selects a more optimized route, which indicates that it has certain advantages in detail processing. Therefore, the SAPO-DDPG model demonstrates stronger competitiveness compared to similar algorithms in the context of single UAV path planning.

In our UAV swarm path planning experiment, we utilize a swarm of five UAVs to simulate collective behavior. Each UAV in the swarm is treated as a dynamic obstacle to the others, allowing for greater adaptability. To enhance the flexibility of the cluster, we intentionally do not incorporate any specific formation for the swarm in this experiment. The performance of each model used in the experiment is detailed in Table 12.

Due to the significant overlap in paths among multiple UAVs, traditional path superposition fails to adequately represent the relative positional relationships between UAVs at different time nodes. Consequently, this study shifts focus from comparing path morphology to three key metrics: total path length, maximum completion time difference, and number of collisions. As shown in Table 12, the SAPO-DDPG model demonstrates distinct advantages in these three metrics compared to other models in swarm path planning. This superiority arises not only from the method's consideration of the UAV swarm's focus during path planning but also from the SAPO approach's treatment of each UAV as an individual entity rather than as part of a whole. This individual-centric dynamic optimization allows for the formulation of a globally optimal solution. As a result, the SAPO-DDPG model significantly enhances individual flexibility and alleviates certain path traffic constraints, leading to its superior performance in UAV swarm path planning compared to similar algorithms.

To verify whether the improvement of the algorithm compared with the existing algorithms is significant, we conducted significance tests on the experimental results of the two models, SAPO-DDPG and TSEB-DDPG. The inspection objectives are the four indicators L_p , T_c , L_t and T_d , and the adopted method is Welch t-tests.

Among them, the statistics to be obtained include: t -value, degrees of freedom df , p -value and effect size Cohen's d . The p -value can be obtained by looking up the table, and the calculation methods of the other three statistics are shown in Eqs. (46)–(48).

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \quad (46)$$

$$df = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}} \quad (47)$$

$$d = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}} \quad (48)$$

where $\bar{X}_i, i = 1, 2$ is the sample means of group i ; $s_i^2, i = 1, 2$ is the unbiased sample variances of group i ; $n_i, i = 1, 2$ is the sample sizes of group i . For the effect size d , $0.2 < |d| < 0.5$ means small effect, $0.5 < |d| < 0.8$ means medium effect and $|d| \geq 0.8$ means large

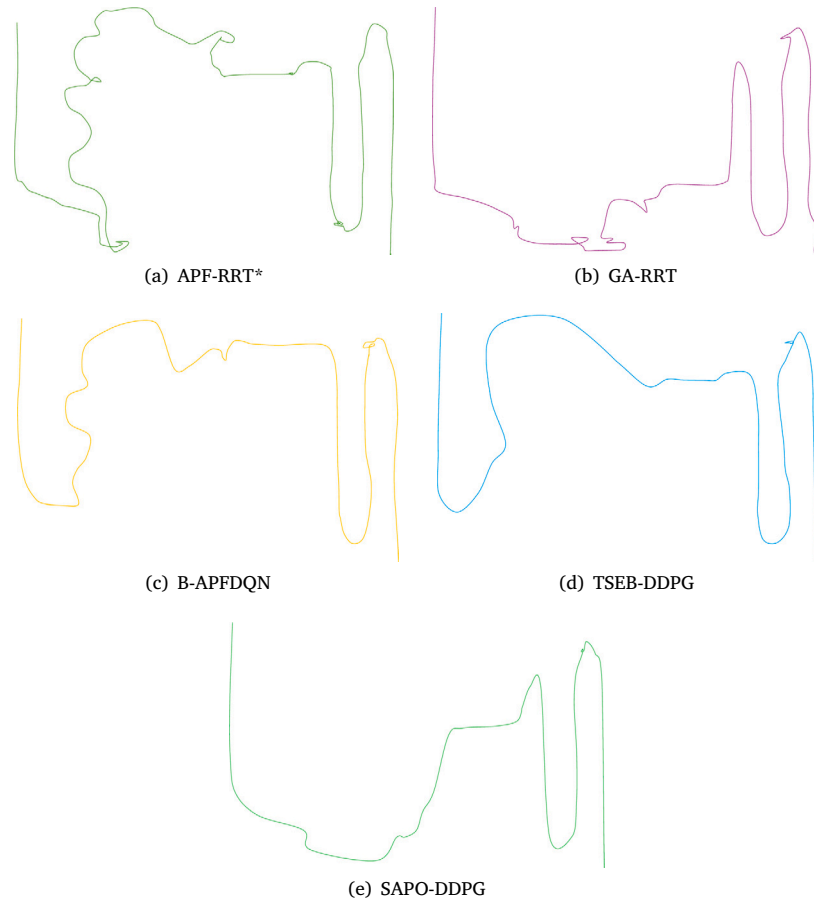


Fig. 14. The path planning results for five models.

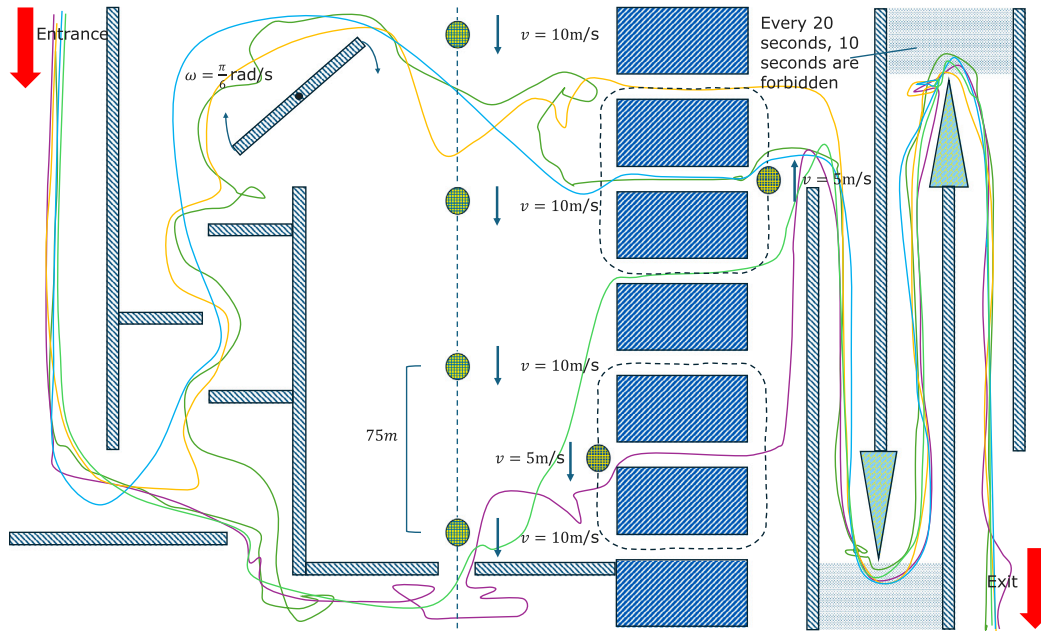


Fig. 15. The path results are overlaid on the test scene.

effect. These equations define our statistical validation framework: Eqs. (46) computes the t-statistic, Eqs. (47) adjusts degrees of freedom, and Eqs. (48) determines effect size. This provides both statistical significance and practical importance assessment.

The test results are shown in Table 13. the results of the Welch's t-test presented in Table 13 provide robust statistical evidence for evaluating the performance improvements of SAPO-DDPG over the TSEB-DDPG baseline. In the single-UAV scenario,

Table 13

Significance test of the experimental data of SAPO-DDPG and TSEB-DDPG.

Statistical magnitude	L_p	T_c	L_t	T_d
t	-6.01	-4.574	-1.632	-2.665
df	17	18	12	17
p	< 0.001	< 0.001	0.128	< 0.05
d	-2.68	-2.05	-0.73	-1.19

the extremely small p-values ($p < 0.001$) for both path length (L_p) and completion time (T_c) allow us to confidently reject the null hypothesis, confirming that the observed performance superiority is statistically significant and not attributable to random chance. More critically, the large effect sizes (Cohen's $d > 2.0$) for both metrics demonstrate that the magnitude of improvement possesses substantial practical significance in real-world applications, far exceeding the conventional threshold for a 'large' effect ($d > 0.8$).

For swarm navigation, the statistical analysis reveals more nuanced but equally meaningful insights. The non-significant result for total path length (L_t , $p = 0.128$) is accompanied by a medium effect size ($d = 0.73$). This pattern suggests that while the reduction in total distance traveled across the swarm exhibits variability that prevents reaching conventional statistical significance thresholds across all experimental runs, there exists a consistent and meaningful trend toward more efficient global routing. The medium effect size indicates that this improvement is nevertheless practically relevant.

Conversely, the statistically significant reduction in maximum time difference (T_d , $p < 0.05$) with a large effect size ($d = 1.19$) provides definitive evidence that SAPO-DDPG achieves superior swarm coordination and synchronization. This metric is particularly critical for mission success in applications where the simultaneous arrival or coordinated operation of all UAVs is operationally essential, such as in coordinated surveillance, formation flying, or simultaneous payload delivery scenarios.

The statistical evidence collectively indicates that SAPO-DDPG not only produces statistically significant improvements in individual UAV performance metrics but also delivers practically meaningful enhancements in swarm coordination under dynamic multi-obstacle environments. The combination of statistical testing with effect size analysis provides a comprehensive validation of our method's effectiveness across both individual and collective performance dimensions.

The experiment of this subsection is to verify whether the proposed SAPO-DDPG model has a positive performance improvement compared with other existing advanced hybrid models. The test scenarios used in the experiment were obtained by making minor modifications to the training scenarios. The experimental results show that the SAPO-DDPG model has achieved a significant improvement in performance compared with other existing models, which reflects the innovation of this study.

5.5. Supplementary experiment

5.5.1. Ablation study

To rigorously evaluate the necessity of our key design choices, we conducted a controlled ablation experiment comparing four model variants in a dynamic multi-obstacle environment (500 m $\times$ 300 m with 10 moving obstacles). All tests used single UAV with randomized start-goal pairs, averaged over 10 Monte Carlo trials.

Four model configurations were tested:

- **Full model:** Our complete model (AHF + SAPO + DDPG).
- **No AHF:** Replaced AHF with adaptive Kalman filters (AKF).
- **No AM:** Removed attention mechanism (AM), retaining only CNN-LSTM.
- **No SAPO:** Removed SAPO's situation assessment, retaining only DDPG.

Table 14Ablation study results (mean $\pm$ std).

Model	Path Length (m)	Collisions	Training time (h)	Time (s)
Full model	1821.4 $\pm$ 13.4	0	29.4 $\pm$ 0.2	103 $\pm$ 8.9
No AHF	1975.8 $\pm$ 17.6	3 $\pm$ 1	29.2 $\pm$ 0.2	121 $\pm$ 9.4
No AM	2017.4 $\pm$ 19.1	3 $\pm$ 1	28.3 $\pm$ 0.2	127 $\pm$ 9.1
No SAPO	2155.4 $\pm$ 22.3	5 $\pm$ 2	21.7 $\pm$ 0.1	152 $\pm$ 10.2

Table 15

Performance metrics for different swarm sizes.

Size	Time (s)	Communication overhead (KB/s)
1	103 $\pm$ 8.9	–
5	172 $\pm$ 16.3	15.6 $\pm$ 2.1
10	246 $\pm$ 25.7	32.4 $\pm$ 3.5
25	278 $\pm$ 35.4	79.9 $\pm$ 5.7
50	331 $\pm$ 47.9	172.3 $\pm$ 12.6

We evaluated four key metrics:

1. **Path length (m):** Total trajectory distance per UAV.
2. **Collisions:** Sum of UAV-obstacle impacts.
3. **Training time (h):** Mean The time consumed by the model during training.
4. **Time (s):** Duration from start to last UAV's arrival.

As shown in Table 14, removing AHF, AM, SAPO degraded performance significantly.

- **AHF's Role:** AKF (No AHF) increased collisions by 300% and reduced safety margins by 21%, confirming that dynamic parameter adaptation is critical for handling unpredictable noise.
- **AM's Role:** In the absence of an attention mechanism, due to the lack of attention to important features, the path was extended by 10.76%, and the collision rate also increased significantly, proving the necessity of the attention mechanism for the model.
- **SAPO's Role:** Without SAPO, paths lengthened by 18.33% due to poor global awareness, while collisions rose sharply (5 vs. 0), proving its necessity for spatiotemporal reasoning.

This experiment verified the necessity of each key module in SAPO-DDPG and confirmed that each sub-model used was beneficial and indispensable.

5.5.2. Scalability verification

To evaluate the computational scalability and communication overhead of SAPO-DDPG, we conducted additional experiments with UAV swarm sizes ranging from 1 to 50 agents. In this experiment, we examined the completion time (from the start to the last UAV leaving the exit) and the communication overhead. The results are summarized in Table 15.

The results show that SAPO-DDPG can maintain a high planning efficiency even in a large group. For the same scenario, after the cluster size was expanded tenfold, the total completion time only increased by about 92% (less than 100%). Meanwhile, as each UAV needs to share basic state information with its surrounding neighbors, the communication overhead approximately increases linearly. The above indicates that in larger-scale UAV swarms, this model still has good planning performance.

This experiment verified the performance changes of the model under a larger cluster scale. The results show that as the scale expands, both the task completion time and the communication load show an upward trend, but they are both within an acceptable range. Therefore, this model has the scalability to meet the requirements of different cluster scales.

Table 16
Comparative performance under extreme conditions.

Metric	500 m × 300 m Environment			1000 m × 600 m Environment		
	Scenario 1	Scenario 2	Scenario 3	Scenario 1	Scenario 2	Scenario 3
SAPO-DDPG						
Path increase (%)	12.3 ± 2.1	8.7 ± 1.8	15.7 ± 2.9	14.2 ± 2.4	11.3 ± 2.1	18.5 ± 3.2
Success rate (%)	100	95	90	94	88	85
TSEB-DDPG						
Path increase (%)	18.7 ± 3.5	14.2 ± 2.9	24.6 ± 4.1	23.5 ± 4.2	19.8 ± 3.7	31.2 ± 5.3
Success rate (%)	78	68	58	72	63	51

5.5.3. Extreme condition testing

To rigorously evaluate the robustness of SAPO-DDPG under diverse adverse conditions, we conducted comprehensive testing across two distinct environmental scales: (1) the standard 500 m × 300 m scene from Section 5.1, and (2) an expanded 1000 m × 600 m scene with higher obstacle density. Three progressively challenging scenarios were executed at each scale, with TSEB-DDPG (Tian et al., 2024) serving as the primary baseline for comparative analysis. Scenario 1 simulated high wind gusts modeled through Dryden turbulence spectra ($L_w = 50$ m, $\sigma_w = 3$ m/s). Scenario 2 introduced sensor degradation including LiDAR range reduction (50 m), GPS packet loss (20%), and IMU drift ($0.1\text{m/s}^2/\text{min}$). Scenario 3 combined these adversities with actuator lag ($\tau = 1.0$ s) and erratic obstacle motion ($\sigma_o = 3.0\text{m/s}^2$).

As systematically quantified in Table 16, SAPO-DDPG consistently outperformed the baseline across all test configurations. The proposed framework maintained success rates above 90% even in Scenario 3 of the standard environment, compared to TSEB-DDPG's deterioration to 58%. This performance advantage extended to the expanded scene, where SAPO-DDPG achieved 85% success under worst-case conditions—a 34% relative improvement over the baseline.

The expanded environment testing revealed two notable trends: First, environmental scale amplified performance differences, with SAPO-DDPG's success rate advantage increasing from 32% to 34% in Scenario 3. Second, wind effects exhibited greater impact in larger spaces, contributing to the 3.8% additional path length increase observed in Scenario 1. Meanwhile, when scaled to a larger scene, the path increment of SAPO-DDPG is smaller compared to the baseline model. This is mainly attributed to the processing of extended spatial features by the CNN-LSTM module, enabling the model to adapt to obstacles of different sizes. These results collectively validate the framework's scalability while demonstrating consistent superiority in preserving operational integrity under conditions exceeding typical uncertainty thresholds.

The purpose of the extreme condition test is to examine the performance differences between the proposed model and the baseline model by increasing the degree of environmental chaos, thereby verifying the environmental adaptability of the SAPO-DDPG model. The experimental results show that with the increase of chaos and site scale, the advantages of the SAPO-DDPG model gradually expand compared with the baseline model. This indicates that SAPO-DDPG has better environmental adaptability, which verifies the advancement of this method.

5.6. Experimental summary

The experimental analysis demonstrates that:

- (1) AHF provides superior smoothing and accuracy, particularly in dynamic environments with complex data distributions;
- (2) The enhanced DDPG achieves faster convergence and improved multi-objective optimization;
- (3) SAPO-DDPG significantly outperforms state-of-the-art methods in dynamic environments, with advantages amplifying as environmental complexity increases.

Table 17 summarizes the characteristics and suitable application scenarios of the evaluated algorithms. While SAPO-DDPG exhibits

longer training times due to its sophisticated architecture, it delivers robust performance in highly chaotic dynamic scenes, making it particularly suitable for mission-critical applications in complex environments.

6. Conclusion

This paper presents SAPO-DDPG, a novel hybrid path planning framework designed to address the complex challenges of UAV swarm navigation in dynamic environments with multiple obstacles. The framework integrates an Adaptive HANTs Filter (AHF), a Situation Assessment Path Optimizer (SAPO), and an enhanced Deep Deterministic Policy Gradient (DDPG) algorithm. This synergistic approach effectively tackles the core difficulties of cluster dynamic path planning under dynamic interference and multi-objective optimization requirements.

Our core theoretical contributions lie in three key innovations. Firstly, the AHF model significantly advances the original HANTs filter by introducing functional relationships to dynamically update critical parameters (error threshold K , maximum deletion points N_e , low-frequency number f_n) during iteration, eliminating reliance on static manual settings. This substantially enhances filtering robustness and trajectory smoothing performance. Secondly, the SAPO model leverages a CNN-LSTM architecture augmented with a channel attention mechanism (placed after the LSTM) to prioritize valuable spatiotemporal features from sensor data for superior situational awareness. It further introduces novel path quality evaluation functions and optimization methods specifically addressing multi-path and multi-bend challenges. Thirdly, we innovate the DDPG algorithm by designing a comprehensive multi-objective reward function (safety, motion stability, traffic efficiency, swarm order) with optimized weights (1:15:20:1) and implementing a prioritized experience replay mechanism based on TD-error, accelerating convergence and improving decision-making in complex scenarios.

These contributions translate into significant practical advantages. Comprehensive experimental results, including rigorous testing under diverse adverse conditions, demonstrate that SAPO-DDPG consistently outperforms state-of-the-art UAV path planning algorithms (including APF-RRT*, GA-RRT, B-APFDQN, and TSEB-DDPG) across critical metrics. Key advantages include: achieving shortest path lengths (individual: 2.1% shorter than TSEB-DDPG, swarm: 4.2% reduced), near-zero collision navigation, and enabling faster mission completion times. The AHF's adaptive filtering and SAPO's attention-driven feature extraction enhance robustness in unpredictable environments (e.g., urban canyons), reducing dependency on pre-mapped global data. The optimized DDPG experience replay supports efficient swarm coordination with manageable communication overhead.

However, the research acknowledges current limitations. The primary constraint is the model's computational complexity, stemming from the CNN-LSTM-Attention structure within SAPO and multi-agent DDPG training. This results in longer training times (approximately 29.4 h in experiments) and imposes higher hardware demands for onboard inference, posing challenges for real-time deployment on resource-constrained or lightweight UAV platforms.

Table 17
The characteristics and application scenarios of several hybrid algorithms.

Algorithm	Dynamic response	Multi-objective optimization	Training efficiency	Application scenario
APF-RRT*	Low	Low	N/A	Static scenes
GA-RRT	Medium	Medium	N/A	Dynamic scenes with regular motion
B-APFDQN	High	High	Medium	Simple and irregular dynamic scene
TSEB-DDPG	High	High	Low	Moderately random dynamic scene
SAPO-DDPG	High	Dynamic	Very low	Highly chaotic dynamic scenes

Future research will focus on addressing these limitations and expanding applicability. Key directions include:

(1) Model lightweighting and acceleration: Exploring techniques like neural network pruning, quantization, or knowledge distillation, or investigating alternative architectures like Spiking Neural Networks (SNNs) to reduce the computational footprint of the SAPO component for edge deployment. Pre-train SAPO independently, share weights among group agents, or adopt course and transfer learning strategies. Use more advanced reinforcement learning algorithms such as SAC and Multi-agent DDPG.

(2) Enhanced sim-to-real transfer: Developing robust transfer learning frameworks incorporating domain randomization to minimize real training time and improve generalization to unseen, highly dynamic environments.

(3) Heterogeneous swarm management: Extending the framework to coordinate mixed teams of UAVs and Unmanned Ground Vehicles (UGVs) with diverse dynamics and objectives, potentially leveraging hierarchical reinforcement learning strategies. These efforts aim to refine SAPO-DDPG for broader autonomous systems applications.

CRediT authorship contribution statement

Hongwei Zhao: Writing – review & editing, Methodology, Formal analysis, Supervision, Project administration. **Ning Yu:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Formal analysis, Data curation. **Yanyan Zhao:** Writing – review & editing, Software, Formal analysis, Data curation. **Juan Feng:** Writing – review & editing, Resources, Investigation. **Yu Wang:** Writing – review & editing, Visualization, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We thank the reviewers for their helpful suggestions, which significantly improved the presentation of this paper.

This research was funded by the Fundamental Research Funds for the Central Universities.

Data availability

Data will be made available on request.

References

Ajith, V.S., Jolly, K.G., 2023. Hybrid optimization based multi-objective path planning framework for unmanned aerial vehicles. *Cybern. Syst.* 54 (8), 1397–1423. <http://dx.doi.org/10.1080/01969722.2022.2157607>.
 Aliabad, F.A., Shojaei, S., Zare, M., Malamiri, H.G., 2024. Investigating the capability of the harmonic analysis of time series (HANTS) algorithm in reconstructing time series images of daytime and nighttime land surface temperature from the MODIS sensor. *Spat. Inf. Res.* 32, 425–439. <http://dx.doi.org/10.1007/s41324-023-00569-3>.

Bo, L., Zhang, T., Zhang, H., Hong, J., Liu, M., Zhang, C., Liu, B., 2024. 3D UAV path planning in unknown environment: A transfer reinforcement learning method based on low-rank adaption. *Adv. Eng. Informatics* 62 (4), 102920. <http://dx.doi.org/10.1016/j.aei.2024.102920>.
 Boulares, M., Fehri, A., Jemni, M., 2024. UAV path planning algorithm based on deep Q-learning to search for a floating lost target in the ocean. *Robot. Auton. Syst.* 179 (1), 1–15. <http://dx.doi.org/10.1016/j.ins.2024.121714>.
 Fan, L., 2023. Agricultural uav crop spraying path planning based on pso-a* algorithm, *inmateh - Agric. Eng.* 71 (3), 625–636. <http://dx.doi.org/10.35633/inmateh-71-54>.
 Fan, J., Chen, X., Liang, X., 2024. ERR-TGA: Expert genetic algorithm with rapidly exploring random tree initialization for multi-UAV path planning. *Expert Syst. Appl.* 8 (8), 1–26. <http://dx.doi.org/10.3390/drones8080367>.
 Fan, Jiaming., Chen, Xia., Wang, Yu., Chen, Xiangmin., 2022. UAV trajectory planning in cluttered environments based on PF-rrt* algorithm with goal-biased strategy. *Eng. Appl. Artif. Intell.* 114, 105182. <http://dx.doi.org/10.1016/j.engappai.2022.105182>.
 Hu, G., Huang, F., Shu, B., Wei, G., 2025. MAHACO: Multi-algorithm hybrid ant colony optimizer for 3D path planning of a group of UAVs. *Inform. Sci.* 694 (1), 1–39.
 Hu, M., Li, X., Ren, Z., Zeng, S., 2024a. UAV 3D path planning based on a* algorithm with improved heuristic function. *Acta Armamentarii* 45 (S1), 302–307. <http://dx.doi.org/10.12382/bgxb.2024.0540>.
 Hu, M., Li, X., Ren, Z., Zeng, S., 2024b. UAV 3D path planning based on a* algorithm with improved heuristic function. *Binggong Xuebao / Acta Armamentarii* 45 (1), 302–307. <http://dx.doi.org/10.12382/bgxb.2024.0540>.
 Huang, T., Fan, K., Sun, W., 2024. Density gradient-RRT: An improved rapidly exploring random tree algorithm for UAV path planning. *Expert Syst. Appl.* 252 (2), 1–11. <http://dx.doi.org/10.1016/j.eswa.2024.124121>.
 Insights, Fortune Business, 2023. Unmanned aerial vehicle market. <https://www.fortunebusinessinsights.com/industry-reports/unmanned-aerial-vehicle-uav-market-101603>. (Accessed 22 March 2025).
 Jiang, X., Huang, B., 2023. Global path planning of fixed-wing UAV based on improved rrt* algorithm. *J. Appl. Sci. Eng.* 26 (10), 1441–1450. [http://dx.doi.org/10.6180/jase.202310.26\(10\).0009](http://dx.doi.org/10.6180/jase.202310.26(10).0009).
 Khalil, H.K., 2002. *Nonlinear Systems*, third ed. Prentice Hall, <http://dx.doi.org/10.1007/978-3-319-57870-1>.
 Kong, F., Wang, Q., Gao, S., Yu, H., 2023. B-APFDQN: A UAV path planning algorithm based on deep Q-network and artificial potential field. *IEEE Access* 11, 44051–44064. <http://dx.doi.org/10.1109/ACCESS.2023.3273164>.
 Kumar, A., Wang, S., Shaikh, A.M., Bilal, H., Lu, B., Song, S., 2024. Building on prior lightweight CNN model combined with LSTM-AM framework to guide fault detection in fixed-wing UAVs. *Int. J. Mach. Learn. Cybern.* 15, 4175–4191. <http://dx.doi.org/10.1007/s13042-024-02141-3>.
 Liu, H., Chen, J., Feng, J., Zhao, H., 2023. An improved RRT uav formation path planning algorithm based on goal bias and node rejection strategy. *Unmanned Syst.* 11 (4), 317–326. <http://dx.doi.org/10.1142/S2301385023500140>.
 Luo, X., Wang, Q., Gong, H., Tang, C., 2024. UAV path planning based on the average TD3 algorithm with prioritized experience replay. *IEEE Access* 12, 38017–38029. <http://dx.doi.org/10.1109/ACCESS.2024.3375083>.
 Pang, S., Chai, Q., Liu, N., Zheng, W., 2024. A multi-objective cat swarm optimization algorithm based on two-archive mechanism for UAV 3D path planning problem. *Appl. Soft Comput.* 167 (2), 1–19. <http://dx.doi.org/10.1016/j.asoc.2024.112306>.
 Rao, J., Xiang, C., Xi, J., Chen, J., Lei, J., Giernacki, W., Liu, M., 2023. Path planning for dual UAVs cooperative suspension transport based on artificial potential field-A* algorithm. *Knowl.-Based Syst.* 277 (2023), 1–20. <http://dx.doi.org/10.1016/j.knsys.2023.110797>.
 Saeed, R.A., Ali, E.S., Abdelhaq, M., Alsaqour, R., Ahmed, F.R.A., Saad, A.M.E., 2024. Energy efficient path planning scheme for unmanned aerial vehicle using hybrid generic algorithm-based Q-learning optimization. *IEEE Access* 12, 13400–13417. <http://dx.doi.org/10.1109/ACCESS.2023.3344455>.
 Schaul, T., Quan, J., Antonoglou, I., Silver, D., 2016. Prioritized experience replay. In: *Proceedings of the International Conference on Learning Representations. ICLR, San Juan, Puerto rico*, <http://dx.doi.org/10.48550/arXiv.1511.05952>.
 Shao, M., Liu, X., Xiao, C., Zhang, T., Yuan, H., 2025. Research on UAV trajectory planning algorithm based on adaptive potential field. *Drones* 9 (2), 79. <http://dx.doi.org/10.3390/drones9020079>.
 Sulieman, M.H., Liu, M., Gursoy, M.C., Kong, F., 2024. Path planning for UAVs under GPS permanent faults. *ACM Trans. Cyber-Physical Syst.* 8 (3), 1–23. <http://dx.doi.org/10.1145/3653074>.

- Tang, J., Liang, Y., Li, K., 2024a. Dynamic scene path planning of UAVs based on deep reinforcement learning. *Drones* 8 (2), 60. <http://dx.doi.org/10.3390/drones8020060>.
- Tang, G., Xiao, T., Du, P., Zhang, P., Liu, K., Tan, L., 2024b. Improved PSO-based two-phase logistics UAV path planning under dynamic demand and wind conditions. *Drones* 8 (8), 356. <http://dx.doi.org/10.3390/drones8080356>.
- Tian, S., Li, Y., Zhang, X., Zheng, L., Cheng, L., She, W., Xie, W., 2024. Fast UAV path planning in urban environments based on three-step experience buffer sampling DDPG. *Digit. Commun. Netw.* 10 (4), 813–826. <http://dx.doi.org/10.1016/j.dcan.2023.02.016>.
- Tu, B., Wang, F., Han, X., 2024. 3D path planning for UAV based on a hybrid algorithm of marine predators algorithm with quasi-oppositional learning and differential evolution. *Egypt. Informatics J.* 28 (1), 1–15. <http://dx.doi.org/10.1016/j.eij.2024.100556>.
- Wang, W., Ye, C., Tian, J., 2024. A 3D UAV path planning method based on multi-strategy improved artificial rabbit optimization algorithm. *Acta Electron. Sin.* 52 (11), 3780–3797. <http://dx.doi.org/10.12263/DZXB.20230382>.
- Wen, C., Dong, H., Xie, W., et al., 2023. Multi-UAVs 3D cooperative curve path planning method based on CEA-GA. *J. Beijing Univ. Aeronaut. Astronaut.* 49 (11), 3086–3099. <http://dx.doi.org/10.13700/j.bh.1001-5965.2021.0787>.
- Xu, H., Niu, Z., Jiang, B., Zhang, Y., Chen, S., Li, Z., Gao, M., Zhu, M., 2023a. UAV trajectory planning based on bi-directional APF-rrt* algorithm with goal-biased. *Drones* 213 (1), 1–12. <http://dx.doi.org/10.1016/j.eswa.2022.119137>.
- Xu, Y., Wei, Y., Wang, D., Jiang, K., Deng, H., 2023b. Multi-UAV path planning in GPS and communication denial environment. *Sensors* 23 (6), 1–15. <http://dx.doi.org/10.3390/s23062997>.
- Yu, X., Jiang, N., Wang, X., Li, M., 2023a. A hybrid algorithm based on grey wolf optimizer and differential evolution for UAV path planning. *Expert Syst. Appl.* 215 (1), 1–13. <http://dx.doi.org/10.1016/j.eswa.2022.119327>.
- Yu, Q., Xu, Z., Duan, N., Xu, M., Cheng, Y., 2023b. Coverage operation path planning of UAV with endurance constraints based on improved ACO. *Acta Aeronaut. et Astronaut. Sin.* 44 (12), 1–13. <http://dx.doi.org/10.7527/S1000-6893.2022.27856>.
- Yu, N., Zhao, H., He, X., 2024. SAPO: A UAV path optimization method through situation assessment. In: 2024 36th Chinese Control and Decision Conference. CCDC, Xi'an, China, pp. 1161–1166. <http://dx.doi.org/10.1109/CCDC62350.2024.10587411>.
- Zhang, J., Guo, Y., Zheng, L., Yang, Q., Shi, G., 2024. Real-time UAV path planning based on LSTM network. *J. Syst. Eng. Electron.* 35 (2), 374–385. <http://dx.doi.org/10.23919/JSEE.2023.000157>.
- Zhang, Z., Wang, S., Chen, J., Han, Y., 2023. A bionic dynamic path planning algorithm of the micro UAV based on the fusion of deep neural network optimization/filtering and hawk-eye vision. *IEEE TRANSACTIONS SYSTEMS, MAN, CYBERNETICS: SYSTEMS* 53 (6), 3728–3740. <http://dx.doi.org/10.1109/TSMC.2023.3233965>.
- Zhao, H., Yan, L., Hou, Z., et al., 2025. Error analysis strategy for long-term correlated network systems: Generalized nonlinear stochastic processes and dual-layer filtering architecture. *IEEE Internet Things J.* 1–14. <http://dx.doi.org/10.1109/JIOT.2025.3578285>, online.
- Zhao, X., Yang, R., Zhong, L., Hou, Z., 2024. Multi-UAV path planning and following based on multi-agent reinforcement learning. *Drones* 8 (1), 18. <http://dx.doi.org/10.3390/drones8010018>.
- Zhong, Huageng, Du, Yu, Liu, Dong, Wang, Minghao, Cong, Ming, Tian, Xiaojing, 2025. A fruit fly-inspired path planning algorithm for unmanned aerial vehicle in underground environments based on low-discrepancy sequences. *Eng. Appl. Artif. Intell.* 156 (Part B), 111250. <http://dx.doi.org/10.1016/j.engappai.2025.111250>.
- Zhong, L., Zhao, J., Luo, H., Hou, Z., 2024. Hybrid path planning and following of a quadrotor UAV based on deep reinforcement learning. In: 2024 36th Chinese Control and Decision Conference. CCDC, Xi'an, China, pp. 1858–1863. <http://dx.doi.org/10.1109/CCDC62350.2024.10587648>.
- Zhou, W., Zhang, C., Tang, W., et al., 2024. A novel modified search and rescue optimization algorithm based on reinforcement learning for UAV path planning. *Control Decis.* 39 (4), 1203–1211. <http://dx.doi.org/10.13195/j.kzyjc.2022.1209>.
- Zhu, Y., Ye, Z., 2025. A game-theoretic cooperative path planning strategy using hybrid heuristic optimization algorithm. *IET Control Theory Appl.* 19 (1), 1–13. <http://dx.doi.org/10.1049/cth2.12766>.